Deep Learning-Based Image Classification of Cat Skin Diseases:

A Comparative Study of ResNet50, MobileNetV2, and EfficientNetB0

A Thesis

Presented to the

School of Information Technology

Mapua Institute of Technology

In Partial Fulfillment

of the Requirements for the Degree

Bachelor of Science in Computer Science

by

Chesney Kayzle A. Bumakil

Kathrin Grace S. Omac

October 2026

TABLE OF CONTENTS

TABLE OF CONTENTS **2**
LIST OF FIGURES **4**
LIST OF TABLES **5**
Chapter 1 **2**
INTRODUCTION **2**
1.1 Background of the Study 2
1.2 Statement of the Problem 3
1.3 Research Questions 4
1.4 Research Objectives 5
1.5 Significance of the Study 5
1.6 Scope and Limitations 6
Chapter 2 **9**
REVIEW OF RELATED LITERATURE & STUDIES **9**
2.1 The Feline Skin Diseases in This Study 9
2.1.1 Flea Allergy Dermatitis 9
2.1.2 Ringworm (Dermatophytosis) 10
2.1.3 Scabies (Notoedric Mange) 10
2.1.4 Healthy Skin and Overall Feline Skin Health 10
2.2 Image Classification of Companion Animal Skin Disease 12
2.3 Transfer Learning and the Selected Architectures 16
2.4 Evaluating Classifiers on Small Image Datasets 18
2.5 Synthesis and Research Gap 19
Chapter 3 **21**
METHODOLOGY **21**
3.1 Research Design 21
3.2 Dataset 22
3.3 Image Preprocessing and Augmentation 26
3.4 Data Partitioning 27
3.5 Model Architectures and Training 29
3.6 Evaluation 34
3.6.1 Classification Performance 34
3.6.2 Statistical Comparison of Classification Accuracy 38
3.6.3 Computational Characteristics 40
3.6.4 Asymptotic Computational Complexity 43
3.6.5 Error Analysis 48
3.7 Comparison of the Classification Models 49
3.7.1 Comparison Conditions 49
3.7.2 Comparison Steps and Metrics 49
3.7.3 Criteria for Judging Performance and Identifying Strengths and Weaknesses 51

Chapter 4 **52**
RESULTS AND DISCUSSION **52**
4.1 Dataset Summary 52
4.2 Training Performance 52
4.3 Objective 1: Classification Performance and Computational Characteristics 55
4.3.1 Overall Classification Performance 55
4.3.2 Per-Class Classification Performance and Disease-versus-Healthy Screening 58
4.3.3 Computational Characteristics 65
4.3.4 Summary of Findings for Objective 1 68
4.4 Objective 2: Comparison of the Three Architectures 69
4.4.1 Descriptive Comparison 69
4.4.2 Statistical Comparison of Classification Accuracy 70
4.4.3 Summary of Findings for Objective 2 72
4.5 Objective 3: Strengths and Weaknesses of Each Architecture 72
4.6 Discussion 75
4.7 Interpretation of the Final Accuracy 77
Chapter 5 **79**
CONCLUSION AND RECOMMENDATION **79**
5.1 Conclusion 79
5.2 Recommendations 81
APPENDICES **83**
Appendix A. Thesis Proposal Defense Approval Form 83
Appendix B. Thesis Proposal Defense Revision List 84
Appendix C. Thesis Proposal Defense Conformity of Revision 85
Appendix D. Thesis Final Defense Revision List 89
Appendix E. Thesis Final Defense Conformity of Revision 90
Appendix F. Expert Dataset Evaluation Form 93
Appendix G. Certificate of Validation 93
REFERENCES **95**

LIST OF FIGURES

Figure 3.1. Study design and procedure. 22

Figure 4.1. ResNet50 training curves (accuracy and loss) across Phase 1 and Phase 2, representative fold 1. Dotted line marks Phase 2 start (epoch 21). 53

Figure 4.2. MobileNetV2 training curves (accuracy and loss) across Phase 1 and Phase 2, representative fold 1. Dotted line marks Phase 2 start (epoch 19). 53

Figure 4.3. EfficientNetB0 training curves (accuracy and loss) across Phase 1 and Phase 2, representative fold 1. Dotted line marks Phase 2 start (epoch 21). 54

Figure 4.4. Cross-validation accuracy (mean ± std) for the three architectures. 57

Figure 4.5. Per-class F1-score by architecture (pooled out-of-fold predictions, n = 360). 61

Figure 4.5a. Confusion matrix, ResNet50 (representative fold 1, n = 72). 62

Figure 4.5b. Confusion matrix, EfficientNetB0 (representative fold 1, n = 72). 62

Figure 4.5c. Confusion matrix, MobileNetV2 (representative fold 1, n = 72). 62

Figure 4.6. Grad-CAM heatmaps for one fold-1 test image of each class (top row: original letterboxed images; lower rows: ResNet50, EfficientNetB0, MobileNetV2). 65

Figure 4.7. Parameters, FLOPs, and model size across the three architectures (model size includes optimizer state). 68

LIST OF TABLES

Table 2.1. Summary of the reviewed image classification studies on companion animal skin disease. **14**

Table 3.1. Dataset composition before and after cleaning. **25**

Table 3.2. Data partitioning in each cross-validation fold. **28**

Table 3.3. Training settings and their basis. **31**

Table 3.4. Best-case and worst-case time complexity of the study's procedures. **46**

Table 4.1. Test-set classification performance (five-fold cross-validation). **56**

Table 4.1a. Per-fold test accuracy (%) by architecture. **57**

Table 4.2. Per-class precision, recall, and F1-score (pooled out-of-fold predictions, n = 360, 90 images per class). **59**

Table 4.2a. Per-class specificity and disease-versus-healthy screening (pooled out-of-fold predictions, n = 360). **60**

Table 4.3. Computational characteristics of the three architectures. **67**

Table 4.4. Pairwise McNemar tests of classification accuracy on the pooled out-of-fold predictions (n = 360), with Holm correction. **71**

‡ Continuity-corrected chi-square (Equation 3.15), the variant the notebook selected because n01 + n10 ≥ 25, with Holm step-down adjustment [54]; values as printed by the notebook and checked there against statsmodels. **72**

Chapter 1

INTRODUCTION

1.1 Background of the Study

Skin conditions are a frequent reason for consultation in small animal general practice [1]. In a review of the clinical records of 3,584 cats attending primary-care practices in England, dermatological disorders were the third most common disorder group, recorded in 10.4% of cats [52]. In cats, several skin diseases share visible signs such as hair loss, redness, scaling, and crusting. In a multicentre study of 502 itchy cats, the common lesion patterns did not point to any single diagnosis, so a full clinical work-up was needed to tell the diseases apart [2]. Some feline skin diseases also matter for people: dermatophytes can infect both animals and humans [15], and feline scabies spreads easily through direct skin contact [4].

A veterinary examination is the reliable way to diagnose these conditions, but access to veterinary care can be limited by cost, distance, and a shortage of staff or equipment [5]. Pet owners increasingly use the internet and mobile phones to look for health information about their animals [6]. An image classifier cannot replace a veterinarian. A reliable one, however, could help owners recognize that a skin problem needs attention. To be useful for this purpose, a classifier must also recognize skin that shows no disease; a model that can only choose among diseases labels every photograph, including one of a healthy cat, as diseased. Because such a tool would most likely run on phones or low cost computers, its computational characteristics, such as model size, computation, and inference time, need to be evaluated together with its classification performance [7].

Convolutional neural networks (CNNs) have already been used to classify photographs of cat skin disease [7][8][9]. These studies used different datasets, disease

classes, and evaluation procedures, and each tested only lightweight architectures. It is therefore still unclear how a lightweight CNN compares with a larger, high capacity CNN when both are trained and tested on the same feline skin images under the same conditions, what computational characteristics each exhibits, and what strengths and weaknesses each shows when both are considered together.

This study focuses on cats for three reasons. First, skin disease is common in cats: dermatological disorders were the third most common disorder group in cats under primary veterinary care in England [52], and flea allergy dermatitis, one of the classes studied, was the most common diagnosis among itchy cats in a multicentre study [2]. Second, some feline skin diseases affect people and other animals: ringworm can infect humans [15], and feline scabies spreads easily through direct skin contact [4]. Third, image classification of skin disease has been studied with a high capacity architecture such as ResNet50 in dogs [3][17], but the feline studies reviewed tested mainly lightweight models and none tested ResNet50 [7][8][9] (Section 2.2), so the comparison made in this study has not yet been made for cats.

1.2 Statement of the Problem

Previous studies show that CNNs can classify feline skin disease images, but their results do not allow the architectures to be compared fairly, for three reasons.

First, none of the feline studies reviewed compared a high capacity architecture such as ResNet50 with lightweight architectures on the same data. Aji et al. [8] and Syahputra and Arifudin [9] evaluated only MobileNetV2, and Aminah et al. [7] compared mainly lightweight models (the largest, DenseNet121, is still far smaller than ResNet50).

Second, the reported accuracies come from single data splits whose test images may not be fully independent of the training images. Aji et al. reported accuracy on a validation

set with no separate test set [8], and Aminah et al. divided an image pool that had already been augmented into training, validation, and test sets [7]. When near duplicate images appear in both training and test data, measured accuracy is inflated [10].

Third, none of the feline studies reviewed reported per class performance and computational characteristics together. Aminah et al. reported parameters, model size, floating point operations (FLOPs), and inference time, but only alongside accuracy [7], while Aji et al. reported per class precision, recall, and F1-score without any computational measure [8].

This study therefore addresses the following problem: when a high capacity CNN (ResNet50) and two lightweight CNNs (MobileNetV2 and EfficientNetB0) are trained and tested under the same procedure on the same photographs of cat skin, covering three skin diseases and healthy skin, what classification performance and computational characteristics does each architecture exhibit, which of the observed differences in classification accuracy are statistically significant, and what strengths and weaknesses does each architecture show when both dimensions are considered together?

1.3 Research Questions

To address the problem stated above, the following research questions were formulated:

RQ1. What classification performance and computational characteristics do ResNet50, MobileNetV2, and EfficientNetB0 exhibit in classifying photographs of cat skin into flea allergy dermatitis, ringworm, scabies, and healthy skin?

RQ2. How do the three architectures differ in these results, and to what extent are the differences in classification accuracy statistically significant?

RQ3. What strengths and weaknesses does each architecture exhibit when its classification performance and computational characteristics are considered together?

1.4 Research Objectives

To answer these research questions, this study aims to conduct a comparative evaluation of ResNet50, MobileNetV2, and EfficientNetB0 for the four-class classification of cat skin photographs, based on their classification performance and computational characteristics. Specifically, this study attempts to:

RO1. Evaluate the classification performance of the three architectures across the four classes, using accuracy, macro-averaged F1-score, the Matthews correlation coefficient, per-class metrics, and disease-versus-healthy screening measures, and their computational characteristics, using the number of parameters, floating point operations (FLOPs), model file size, and inference time;

RO2. Compare the three architectures on these results and test the pairwise differences in classification accuracy for statistical significance; and,

RO3. Identify the strengths and weaknesses of each architecture across classification performance and computational characteristics.

1.5 Significance of the Study

For researchers in veterinary image classification, the study provides a controlled comparison of one high capacity and two lightweight CNNs on feline skin images, together with an evaluation procedure (near-duplicate checking, cross-validation, significance testing, and a disease-versus-healthy screening measure) that can be reused in later studies. For developers of pet health tools, it reports the classification performance and computational characteristics of the three architectures side by side and identifies the strengths and

weaknesses of each, which is the information needed to judge which architecture suits a given device and purpose. Because the models include a healthy skin class, the study also shows how often each architecture mistakes a diseased cat for a healthy one, the error that matters most for a screening tool. For pet owners and veterinarians, the study is a step toward screening tools; it does not produce a diagnostic tool.

1.6 Scope and Limitations

The study covers image classification of cat skin photographs into four classes: three skin diseases (flea allergy dermatitis, ringworm, and scabies) and healthy skin. It compares three CNN architectures (ResNet50, MobileNetV2, and EfficientNetB0) on 360 publicly available photographs, 90 per class. Skin diseases other than the three studied are not represented, so an image of another condition is still assigned to one of the four classes. Given a photograph, each model outputs a probability for each of the four classes, and the class with the highest probability is its prediction (Section 3.5, Equation 3.2); the model labels the photograph as a whole and does not mark where a lesion is. The architectures are evaluated on two dimensions. The first is classification performance: accuracy, macro-averaged F1-score, Matthews correlation coefficient (MCC), per-class precision, recall, and F1-score, confusion matrices, and disease-versus-healthy screening measures. The second is computational characteristics: number of parameters, FLOPs, model file size, and inference time. Pairwise differences in classification accuracy are tested for statistical significance with McNemar's test, and the strengths and weaknesses of each architecture are identified from both dimensions together. The study classifies photographs; it does not cover lesion localization, severity grading, clinical diagnosis, treatment, or deployment, and it does not select a model for clinical use.

The results should be read with the following limitations. The dataset is small (360 images, with 72 test images per fold). The labels come from public repositories and were reviewed during data preparation. A veterinarian evaluated the dataset (Section 3.2, Appendices F and G), judged it valid with corrections, noted one image in the Ringworm folder as atopic dermatitis in a dog, a concern later resolved in an in-person discussion with the validator, and rated the clarity of the photographs low. Splitting was done per image because the sources do not identify individual cats, so photographs of the same cat may appear in different folds, which can inflate accuracy [11]. There is no external or clinical test set. Near-duplicates were searched for with a perceptual hash, which may miss flipped or rotated copies of the same photograph. All three models used one fixed set of hyperparameters, so the results do not represent each architecture at its best possible setting. Per class metrics, the disease-versus-healthy screening measure, and the McNemar tests were computed on the 360 pooled out-of-fold predictions, but the pooled McNemar tests do not account for the variation between the five separately trained models of each architecture. The training curves, confusion-matrix figures, and Grad-CAM examples come from one representative fold. Inference time was measured on a Google Colab T4 GPU, and the model files saved during training include optimizer state (inference-only sizes are reported separately), so these inference-time and file-size figures do not represent deployment on a specific device. Inference times were measured in eager execution, which adds framework overhead; a graph-mode timing is reported for comparison. Breed, coat type, coat color, sex, and age were not recorded in the image sources, so performance across different types of cats was not examined (Section 2.1.4). Coat color is a particular concern because lesions such as redness, scaling, crusting, and hair loss look different against light and dark fur and may be harder to see on dark or patterned coats; because the number of images of each coat color in each class is unknown, the models may perform unevenly across coat colors, and part of what

they learned may reflect coat color rather than the skin condition. Finally, the healthy skin photographs may differ from the disease photographs in framing (for example, whole-cat photographs versus close-ups of lesions), which could make the healthy class easier to recognize for reasons unrelated to the skin itself (Section 4.3.2).

Chapter 2

REVIEW OF RELATED LITERATURE & STUDIES

2.1 The Feline Skin Diseases in This Study

The three disease classes were chosen because they are the disease classes shared by the two earlier feline CNN studies that this study builds on. Aji et al. classified fungal infection, flea infestation, scabies, and healthy skin [8], and Aminah et al. classified ringworm, acne, flea, and scabies at two severity levels, plus healthy skin [7]. Both studies also included a healthy skin class, which this study adopts (Section 2.1.4). Using the classes they share keeps this study comparable with that work. The disease classes were also available in the public repositories used [12][13]. The selection was not based on a ranking of disease frequency, and the study does not claim that these are the three most common feline skin diseases. The clinical relevance of each class is described below.

2.1.1 Flea Allergy Dermatitis

Flea allergy dermatitis, also called flea bite hypersensitivity, was the most common diagnosis in a multicentre study of 502 itchy cats, accounting for 29% of cases [2]. Cats with allergic skin disease show one or more of four lesion patterns: miliary dermatitis (small crusted bumps), eosinophilic lesions, hair loss caused by over-grooming, and scratches on the head and neck [2][14]. None of these patterns was specific to one diagnosis [2], and flea allergy must be ruled out before other allergic skin diseases can be diagnosed [14]. For this study, this means flea allergy dermatitis is not expected to be perfectly separable from other conditions by appearance alone.

Earlier studies named this class differently: Aji et al. used "flea infestation" [8], while a public feline dataset used by Syahputra and Arifudin labels it "flea allergy" [9]. This study uses "flea allergy dermatitis" throughout, following the label used in its source data.

2.1.2 Ringworm (Dermatophytosis)

Ringworm is a superficial fungal infection of the skin and hair. Microsporum canis is one of the main species that infect cats [15], and dermatophytes can infect both animals and people [15]. The most consistent sign is one or more round patches of hair loss with scaling; some cats show the classic ring shape with a healing centre and crusts at the edge. Lesions are usually uneven in distribution, and itching is often mild or absent [15]. Because ringworm also causes hair loss and crusting, it can resemble flea allergy dermatitis in photographs.

2.1.3 Scabies (Notoedric Mange)

Feline scabies is caused by the burrowing mite Notoedres cati and spreads through direct skin contact [4]. Its most common sign is thickened skin with grey crusts and scale, with intense itching of the ear margins, face, and head [4]. Its tendency to affect the head and ears gives it a more typical location than the other two classes.

Taken together, the three diseases share hair loss and crusting, and flea allergy dermatitis and ringworm in particular can look alike. Earlier feline studies found the same kind of overlap: the fungal class had the lowest recall in Aji et al., with errors going to flea and scabies [8], and fungus and scabies were confused before fine-tuning in Syahputra and Arifudin [9]. This is why the study reports results for each disease and not only overall accuracy (Section 2.4).

2.1.4 Healthy Skin and Overall Feline Skin Health

Skin disease is a common part of overall feline health. In a review of the clinical records of a random sample of 3,584 cats attending 91 primary-care veterinary practices in England, dermatological disorders were the third most prevalent disorder group, after dental conditions and traumatic injury, affecting 10.4% of cats (95% CI, 9.2 to 11.7%) [52]. A later study of 18,249 cats under primary veterinary care in the United Kingdom in 2019 reported that female cats were more likely than male cats to be diagnosed with over-grooming and flea bite hypersensitivity [53]. These studies show that skin problems are among the conditions most often brought to a veterinarian, and that flea-related allergy, one of the classes in this study, is a routine clinical problem.

The cats affected also differ in their characteristics; in the UK study, for example, over-grooming and flea bite hypersensitivity were diagnosed more often in female than in male cats [53]. The feline image classification studies reviewed did not record such characteristics: none reported the breed, coat type, sex, or age of the cats photographed [7][8][9], and Aminah et al. stated only that their images were selected for diversity in lighting, pose, background, fur color, and lesion appearance [7]. The public sources used in this study do not record these characteristics either, so the dataset cannot be described by type of cat and performance across types of cats could not be examined (Section 1.6). Recording them is recommended for future datasets (Section 5.2).

A screening tool must also recognize when no disease is visible. In this study, the healthy skin class represents photographs of cats without the visible signs described above, namely patchy hair loss, redness, scaling, and crusting [2][4][15]. Earlier feline studies included a healthy class for the same reason [7][8], and Hwang et al. framed dog skin classification as disease versus healthy skin [3]. Without such a class, a classifier must assign one of the diseases to every photograph, including photographs of healthy cats. Adding the

class therefore allows the study to measure how often each model mistakes a diseased cat for a healthy one, which is the more costly error for a screening tool.

2.2 Image Classification of Companion Animal Skin Disease

Earlier computer tools for feline skin disease were expert systems that asked the owner to enter observed symptoms and then applied rules to suggest a disease [16]. Such systems depend on the owner recognizing symptoms correctly. Image based classification avoids this step by learning directly from photographs, and recent work has used CNNs for this purpose. Table 2.1 summarizes the studies reviewed.

In dogs, Hwang et al. trained four CNNs from random weights on images of 95 dogs to separate each of three skin diseases from healthy skin [3]. They reported accuracy together with the Matthews correlation coefficient (MCC), sensitivity, and specificity because accuracy alone hid models that were biased toward one class. Manoj et al. applied ResNet50 with transfer learning to six dog skin classes, including ringworm, and reported 92.3% accuracy [17]. For pets more generally, Taufiqoh and Purnamasari found that on only 72 phone images a small custom CNN performed better than ResNet50 and VGG16 [18], which shows that larger models do not automatically perform better on very small datasets.

In cats, three studies used transfer learning. Aji et al. trained MobileNetV2 on 1,600 images of four classes and reached 91.8% accuracy on a validation set; the same network trained without pretrained weights reached 80.0% [8]. Syahputra and Arifudin trained MobileNetV2 on 1,543 images of six classes and reported that fine-tuning the upper layers raised accuracy from about 83% to 92% [9]. Aminah et al. compared five lightweight

architectures on 720 images in nine classes and reported test accuracies of 99.38% to 99.79% for MobileNetV2, EfficientNetB0, and EfficientNetV2B0, together with parameters, model size, FLOPs, and CPU inference time [7].

Framework of the base studies. This study builds most directly on Aji et al. [8] and Aminah et al. [7], whose disease classes it shares (Section 2.1). Both followed the same four-step framework. In the first step, data collection, Aji et al. combined 1,600 images from one Kaggle and one Roboflow dataset into four balanced classes [8], and Aminah et al. compiled 720 images in nine classes from Kaggle, Roboflow, and Google Images [7]. In the second step, preprocessing, the images were resized and augmented: Aji et al. applied rotation, shifting, zooming, and horizontal flipping during training [8], while Aminah et al. augmented the images before splitting them, which enlarged the pool to 5,760 images [7]. In the third step, training, an ImageNet-pretrained network received a new classification head; Aji et al. trained the head first and then fine-tuned the last layers of MobileNetV2 [8], and Aminah et al. trained and compared five networks, mostly lightweight ones [7]. In the fourth step, evaluation, Aji et al. reported accuracy, per-class precision, recall, and F1-score, and a confusion matrix on an 80/20 split without a separate test set [8], and Aminah et al. reported test accuracy together with parameters, model size, FLOPs, and CPU inference time [7].

This study keeps the same four steps (Section 3.1, Figure 3.1) and changes five things: it adds ResNet50 as a high capacity architecture (Section 2.3); it checks for near-duplicate images before any split and augments only training images within each fold (Sections 3.2 and 3.3); it replaces the single split with stratified five-fold cross-validation and an inner validation set (Section 3.4); it adds MCC, disease-versus-healthy screening, confidence intervals, and McNemar tests with Holm correction (Section 3.6); and it trains and tests all three architectures under identical settings (Section 3.5).

These studies show that transfer learning with lightweight CNNs can classify feline skin images. They also share three features that limit what their results can tell a reader choosing between architectures. Each used a single split of the data. None removed near duplicate images or tested whether differences between models were statistically significant. And none compared a lightweight model with a high capacity model such as ResNet50 on feline data.

Table 2.1. Summary of the reviewed image classification studies on companion animal skin disease.

Study	**Animal and classes**	**Images**	**Models**	**Evaluation design**	**Metrics**	Computational measures
Hwang et al. [3]	Dog; 3 diseases, each vs healthy (binary)	95 dogs (normal and multispectral)	ResNet50, InceptionV3, DenseNet121, MobileNetV3Small (no transfer learning)	Train, test, and separate validation images	Accuracy, AUC, MCC, sensitivity, specificity	None

Manoj et al. [17]	Dog; 6 classes incl. ringworm and healthy	Not reported	ResNet50 (transfer learning)	Separate test set	Accuracy, precision, recall, F1, AUC	None
Taufiqoh and Purnamasari [18]	Pets; ringworm, scabies, ear mite	72	Custom CNN vs ResNet50, VGG16	Test set	Accuracy, precision, recall	None
Aji et al. [8]	Cat; fungal, flea, scabies, healthy	1,600	MobileNetV2 (transfer learning)	Single 80/20 split; no test set	Accuracy, precision, recall, F1, confusion matrix	None
Syahputra and Arifudin [9]	Cat; 6 classes incl. flea allergy, ringworm, scabies	1,543	MobileNetV2 (transfer learning, fine-tuning)	Single fixed split	Accuracy, precision, recall, F1, confusion matrix	None

Aminah et al. [7]	Cat; 4 diseases at 2 severities, plus healthy	720 (5,760 after offline augmentation)	DenseNet121, MobileNetV2, MobileNetV3, EfficientNetB0, EfficientNetV2 B0	Single split of the augmented pool	Accuracy	Parameters, size, FLOPs, CPU time
This study	Cat; flea allergy dermatitis, ringworm, scabies, healthy skin	360 (no near-duplicates detected by pHash)	ResNet50, MobileNetV2, EfficientNetB0 (transfer learning)	Stratified 5-fold CV with inner validation	Accuracy, macro P/R/F1, MCC, specificity, disease-vs-healthy screening, confusion matrix, CIs, McNemar	Parameters, FLOPs, size, GPU time

2.3 Transfer Learning and the Selected Architectures

Convolutional neural networks learn image features through stacked convolution and pooling layers and are the standard approach to image classification [45]. Transfer learning starts from a network already trained on a large image collection, usually ImageNet [19], and adapts it to a new task. Across four medical imaging tasks, Tajbakhsh et al. found that

fine-tuned pretrained CNNs performed as well as or better than CNNs trained from scratch and were more robust when training data were few; they also found that the best number of layers to fine-tune differed between tasks [20]. The feline studies agree with the first point: pretrained MobileNetV2 outperformed the same network trained from scratch [8], and fine-tuning the upper layers improved accuracy further [9]. Fine-tuning can also cause overfitting when data are very limited [21]. For these reasons the study trains the new classification layers first and then fine-tunes only part of each network, with early stopping (Section 3.5). The three architectures were selected to represent one high capacity design and the two lightweight designs used in earlier feline work.

ResNet50 uses residual (shortcut) connections that make very deep networks trainable [22]. It is the largest of the three models in this study and serves as the high capacity reference. It has been used for dog skin disease [3][17] but not in any of the feline studies reviewed, which is why it is needed for the comparison.

MobileNetV2 uses inverted residual blocks built from depthwise separable convolutions, which reduce computation for mobile devices [23]. It is the model used most often in the feline studies [7][8][9].

EfficientNetB0 is the smallest member of a family that scales network depth, width, and input resolution together [24]. It reached 99.78% test accuracy in the feline comparison of Aminah et al., although it ran more slowly on a CPU than MobileNetV2 (140 ms versus 80 ms per image) [7].

EfficientNetV2B0, the best model in Aminah et al. [7], and other families such as DenseNet were not included. This scoping decision kept one high capacity and two lightweight models; it is a limitation of the study. A study on human skin images offers one point of comparison: on more than 27,000 phone images of 51 human skin conditions,

ResNet-50 and EfficientNet-B0 performed about the same and MobileNetV2 performed clearly worse [21]. Because those are human images, this result is used only to show that the ranking of these architectures depends on the data, not as evidence about cats.

2.4 Evaluating Classifiers on Small Image Datasets

Independent test data. A test result is only meaningful if the test images are not repeated in training. When near duplicate images appear in both training and test sets, accuracy is overstated; removing such duplicates from CIFAR test sets lowered measured accuracy by 9% to 14% [10]. A related problem occurs when several records from the same subject fall in both sets: splitting by record instead of by subject often overestimates accuracy [11].

Resampling. With few images, a single train and test split gives an estimate that depends heavily on which images happen to be in the test set. Stratified k-fold cross-validation tests every image once while keeping class proportions in each fold, and Kohavi recommends it for accuracy estimation and model selection [25]. If the same data are used both to tune a model and to report its error, the reported error is biased [26], so data used for decisions during training, such as early stopping [27], must be kept apart from the test data.

Performance metrics. Accuracy summarizes overall correctness but can look good even when one class is handled poorly, especially with unequal class sizes [28][29]. Precision, recall, and F1-score computed for each class, and their unweighted (macro) averages, show how well each class is handled [30]. MCC uses all cells of the confusion matrix and is high only when all classes are predicted well [28]; Gorodkin extended it to more than two classes [31]. Hwang et al. used MCC for this reason in dog skin classification [3].

Comparing models. When each model is trained once and tested on the same images, McNemar's test is the recommended test among those compared by Dietterich, because it has an acceptable false positive rate in that setting [32]. Two further points guide how results are read. Whether two confidence intervals overlap is not a formal test of difference [33], and a result that is not statistically significant does not show that two models are equal [34]. When several models are tested on the same images, their accuracies are paired rather than independent observations; Dietterich found that tests that ignore this pairing, such as the difference-of-proportions test, can have an elevated false positive rate [32]. When several pairwise tests are made, the chance of at least one false positive also rises, so a correction for multiple comparisons is usually applied, such as the Holm step-down procedure [54].

Computational cost. FLOPs and parameter counts describe model size and computation, but actual speed also depends on memory access and on the hardware [35]. For this reason, Aminah et al. reported parameters, model size, FLOPs, and CPU inference time together [7]. Reporting hardware-independent measures (parameters and FLOPs) together with measured ones (model file size and inference time) gives a fuller picture of what each architecture requires. It also allows classification performance to be weighed against these requirements rather than judged on its own. Asymptotic analysis complements these measurements: it describes how the number of operations grows with the size of the input, and best-case and worst-case analysis bounds that number over all inputs of a given size [50].

Checking what the model uses. Gradient-weighted class activation mapping (Grad-CAM) highlights the image regions that most influenced a prediction and can reveal when a model relies on irrelevant cues [36].

2.5 Synthesis and Research Gap

The reviewed studies establish that transfer learning with CNNs can classify feline skin disease photographs, and that lightweight models such as MobileNetV2 and EfficientNetB0 can reach high reported accuracy [7][8][9]. Within the studies reviewed, three things are not yet addressed.

1. No feline study compared a high capacity model such as ResNet50 with lightweight models on the same images.

2. Feline results come from single splits, validation only evaluation, or splits made after augmentation, without removing near duplicates and without significance tests. Section 2.4 shows why these designs can overstate accuracy or hide uncertainty [10][25][32].

3. No feline study reported per class performance together with computational characteristics, so the strengths and weaknesses of architectures across both dimensions have not been described.

This study addresses these points in three steps. First, it evaluates ResNet50, MobileNetV2, and EfficientNetB0 on the same near-duplicate-checked photographs of three skin diseases and healthy skin. For each architecture it reports classification performance (accuracy, macro-averaged F1-score, MCC, per-class metrics, confusion matrices, and disease-versus-healthy screening) together with four computational characteristics (RQ1 and RO1; points 1 and 3). Second, it compares the architectures under stratified five-fold cross-validation, with bootstrap confidence intervals and pairwise McNemar tests of classification accuracy (RQ2 and RO2; points 1 and 2). Third, it considers both dimensions together to identify the strengths and weaknesses of each architecture (RQ3 and RO3; point 3). This review was not a systematic search, so the gap is stated for the studies reviewed and not for all published work.

Chapter 3

METHODOLOGY

3.1 Research Design

The study uses a comparative experimental design with one varied factor, the CNN architecture, at three levels: ResNet50, MobileNetV2, and EfficientNetB0. Everything else is held constant: the images, the cross-validation folds, the preprocessing and augmentation, the classification head, the training schedule, and the hyperparameters. Holding these conditions constant means that differences in results can be attributed to the architecture. Two groups of outcomes are measured for each architecture to answer RQ1 and RO1. The first is classification performance (Section 3.6.1). The second is computational characteristics (Sections 3.6.3 and 3.6.4). The architectures are then compared descriptively and through pairwise statistical tests of classification accuracy (RQ2 and RO2, Sections 3.6.2 and 3.7). Finally, the two groups of outcomes are considered together to identify the strengths and weaknesses of each architecture (RQ3 and RO3, Section 3.7.3).

The study follows an input-process-output framework. The input is the set of 360 cat skin photographs in four classes. The process consists of data cleaning and preprocessing, partitioning by stratified five-fold cross-validation, and two-phase transfer learning of ResNet50, MobileNetV2, and EfficientNetB0 for four-class classification. It is followed by evaluation of classification performance, measurement of computational characteristics, and statistical comparison of classification accuracy. The output is a comparative evaluation of the three architectures and a statement of the strengths and weaknesses of each. Figure 3.1 shows the full procedure.

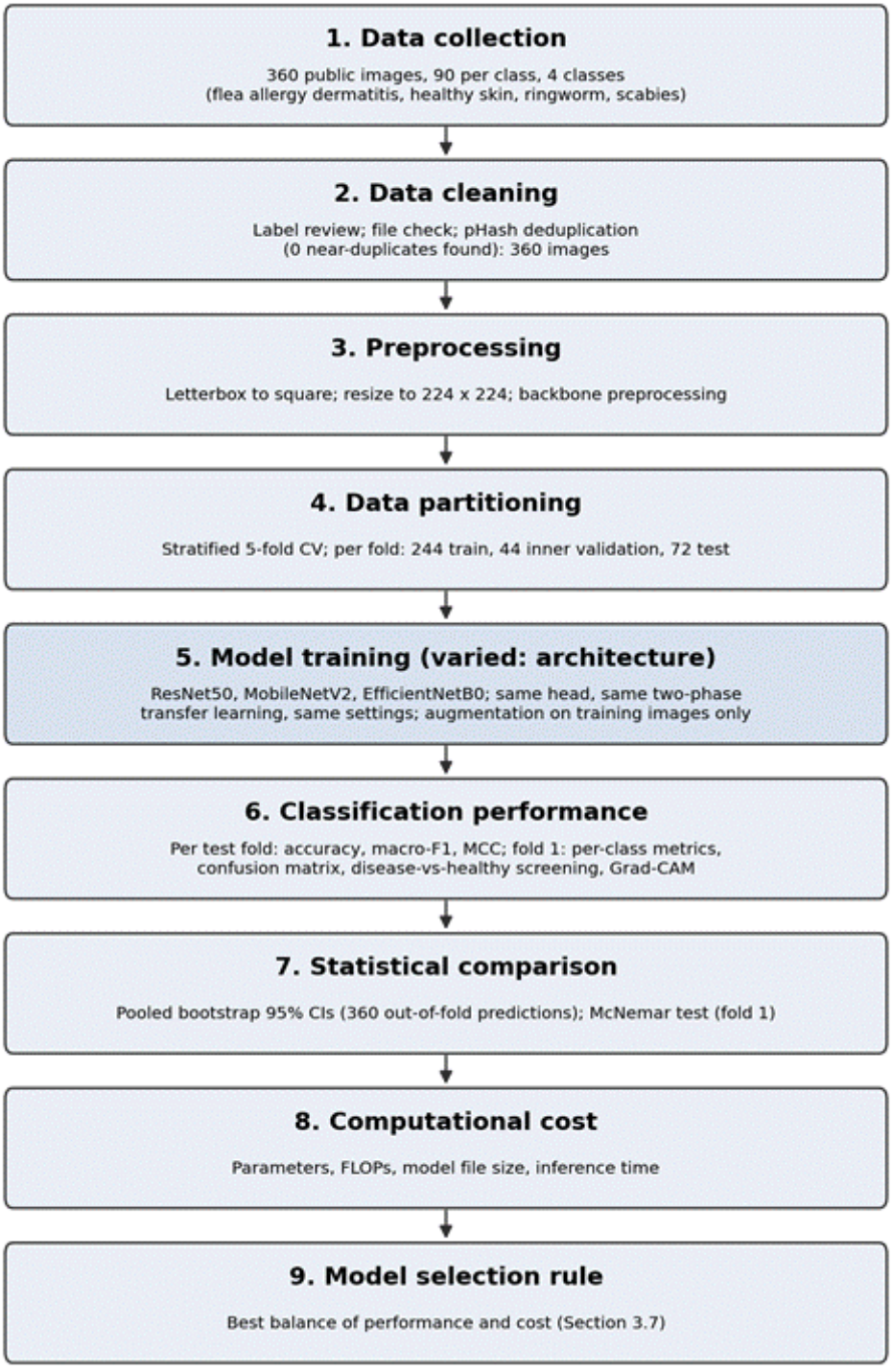


Figure 3.1. Study design and procedure.

3.2 Dataset

All images came from two public repositories, Kaggle [12] and Roboflow Universe [13]; the healthy skin images came from the Kaggle dataset [12], and no images were taken

from other websites. Datasets on these platforms are uploaded by their users, and the platforms do not verify the labels. The Kaggle dataset [12] lists K. Beohar, D. Battawar, and A. Vahreza as contributors; a related Kaggle dataset, Cat Skin Disease V2, published by the user adityavahreza, was one of the two sources used by Aji et al. [8]. The Roboflow dataset [13] was published by the user Tain in 2022 and is not named by any of the studies reviewed. Syahputra and Arifudin used a different Roboflow dataset [9], and Aminah et al. compiled their own dataset from Kaggle, Roboflow, and Google Images [7]. Because the earlier feline studies drew on the same platforms, the data are of the same kind as theirs; because the platforms do not establish label accuracy, the credibility of the labels was checked in two ways, by the researchers' own label review and by an independent veterinary evaluation (see Cleaning and Expert validation below; Appendices F and G). The notebook reads the images from four class folders named Flea Allergy Dermatitis, Healthy Skin, Ringworm, and Scabies, and sets the number of classes from these folders. Because images are stored by class and not by source, the number contributed by each source cannot be determined from the notebook.

Type of data. The data are color (RGB) photographs of cats stored in common image formats; the notebook accepts JPEG, PNG, BMP, and WebP files. Each image carries one class label, given by the folder in which it is stored. There are no bounding boxes, lesion masks, severity grades, or cat identifiers. Because the images were collected from several public sources, they vary in framing, lighting, background, coat color, and resolution. The expert evaluation raised a concern about one image in the Ringworm folder (see Expert validation below).

Classes and size. The four classes and the reasons for them are given in Section 2.1. The final dataset contains 90 images per class (360 in total), as recorded when the notebook

letterboxed the images. The size of the dataset was limited by availability: 90 usable images per class were all that could be gathered from these public sources, which also gave a balanced dataset. For comparison, Aminah et al. also used an equal number of images per class (80) [7].

Cleaning. Before the images were loaded into the notebook, the researchers reviewed the labels in all four class folders, including Healthy Skin, and corrected them where needed. This review was not done by a veterinarian, and the notebook contains no record of it. The dataset was then evaluated by a veterinarian (see Expert validation below). The notebook checked for images lost because two source files shared a name with different extensions and found none. Near duplicate images were then searched for with a perceptual hash (pHash) computed with the imagehash library (hash size 8, giving a 64-bit hash). Two images were compared with the Hamming distance between their hashes:

$$d_H(h_a, h_b) = \sum_{i=1}^{64} [\, h_{a,i} \neq h_{b,i} \,] \qquad (3.1)$$

Here h_a and h_b are the hashes of images a and b, h_a,i is bit i of hash h_a, and the bracket equals 1 when the two bits differ and 0 when they match. The distance therefore counts differing bits: 0 means identical hashes and 64 means every bit differs. Perceptual hashes are designed so that visually similar images give similar hashes, and they are evaluated by comparing hashes in this way [37]. Images with d_H of 5 or less are treated as copies, and only one image from each group is kept. The threshold of 5 was a design choice and was not tuned. This step matters because it decides which images are removed before splitting, which prevents near copies from appearing in both training and test data [10]. In the final run, all 64,620 pairs of the 360 images were compared (calculated: 360 × 359 / 2); no near-duplicate group and no group spanning two classes was found, so no image was

removed. Perceptual hashes are not designed to match flipped or rotated copies, so augmented copies of one photograph, which some Roboflow dataset versions include, would not necessarily be detected. Table 3.1 shows the resulting dataset.

Table 3.1. Dataset composition before and after cleaning.

Class	Collected	After cleaning
Flea allergy dermatitis	90	90
Healthy skin	90	90
Ringworm	90	90
Scabies	90	90
Total	360	360

Expert validation. The dataset used in the experiments reported in Chapter 4 was evaluated by a veterinarian, Laiza Baning, DVM, Resident Veterinarian at Paw Haus Vet Clinic, using the Dataset Evaluation Form in Appendix F. The signed Certificate of Validation is in Appendix G. The validator reviewed the photographs in the four class folders and rated nine statements on a five-point scale (1 = strongly disagree, 3 = neutral, 5 = strongly agree). For label consistency, the Scabies and Healthy Skin folders were each rated 5 and the Flea Allergy Dermatitis and Ringworm folders were each rated 4. For image quality, the statement that the affected skin area is visible in most photographs was rated 4, but the statement that the photographs are clear enough to judge the condition of the skin was rated 2, the only rating below neutral. The three overall statements, that the four categories can

generally be told apart from a photograph, that most photographs are correctly labeled, and that the dataset is suitable for a research study on classifying cat skin conditions, were each rated 4. The validator's overall judgement was "valid with corrections." The photographs of healthy skin and of scabies were named as the most reliable part of the dataset. The main labeling concern was in the Ringworm (dermatophytosis) folder, where the form notes one image as atopic dermatitis in a dog. This concern was subsequently discussed with the validator in person and resolved. To make the dataset more reliable, the validator recommended clearer images.

Effect on this study. Table 3.1 is unchanged, and the results in Chapter 4 come from the 360-image dataset that the validator evaluated. The expert evaluation served as an independent check of the labels assigned during data preparation rather than as a relabeling of the dataset. The low clarity rating reflects the limitations of photographs collected from public online sources and is considered in the interpretation of the results (Section 4.7).

Limitations of the data. The dataset is small and the images do not identify individual cats. The veterinarian's evaluation judged the dataset valid with corrections. It raised a concern about one image in the Ringworm folder, later resolved with the validator, and rated image clarity below neutral. The four classes are balanced (90 images each), so class imbalance is not a concern in the final dataset. These limitations are carried into the interpretation of results (Section 4.7).

3.3 Image Preprocessing and Augmentation

Letterboxing and resizing. Each image was padded to a square with neutral gray (RGB 114, 114, 114) while keeping its original shape, saved at 384 x 384 pixels, and resized to 224 x 224 pixels for training. The size 224 x 224 matches the input size of the ImageNet pretrained weights [7]. Padding was chosen over stretching so that round lesions, such as

ringworm patches [15], keep their shape. The evidence on this choice is mixed: padding did not reduce CNN accuracy in one study [38], and on document images no alternative outperformed plain stretching [39]. Letterboxing is therefore a design choice and not a proven accuracy gain; its effect was not tested.

Backbone preprocessing. Each architecture received its own Keras input preprocessing: mean subtraction in BGR order for ResNet50, scaling to [−1, 1] for MobileNetV2, and raw [0, 255] values for EfficientNetB0, which normalizes inputs inside the model. The Keras documentation states that each pretrained application expects its own input preprocessing, and that EfficientNet includes its normalization inside the model [40].

Augmentation. During training only, each image was randomly rotated by up to 30 degrees, shifted by up to 10%, zoomed by up to 15%, and flipped horizontally and vertically. The inner validation and test images were not augmented. These operations are standard for small image datasets [41] and are of the same kind as those used by Aji et al. (rotation 20 degrees, shift 10%, zoom 20%, horizontal flip) [8]; the exact ranges were design choices and were not tuned. Augmentation was done on the fly inside each fold, not before splitting, so that modified copies of a test image could never appear in training [10].

3.4 Data Partitioning

The 360 images were divided with stratified five-fold cross-validation (random seed 42). In each round, one fold (20%, 72 images, 18 per class) was the test set and the other four folds (288 images) were used for training. Stratification keeps the class proportions the same in every fold [25]. Cross-validation was chosen over a single split, such as the 70/15/15 split planned at the proposal stage, because, with 360 images, a single test set would be small (a 15% test set would hold only 54 images, about 13 or 14 per class) and its result would depend on which images it contained; cross-validation tests every image exactly once and shows how

much results vary between folds. As a result, all available data were used for testing: every one of the 360 images, including every healthy skin image, served as a test image exactly once.

The study used five folds (N_SPLITS = 5 in the notebook). Five and ten are the fold counts most often recommended, because they have been shown empirically to give test error estimates that suffer neither from excessively high bias nor from very high variance [42]; Kohavi recommends ten for model selection [25]. Five folds suit the size of this dataset: with 360 images, five folds give 72 test images per fold (18 per class), whereas ten folds would give only 36 (9 per class), so each fold's per-class results would rest on very few images. Five folds also require 15 model trainings (3 architectures × 5 folds, each trained in two phases, or 30 calls to the training function), against 30 for ten folds.

Within the training portion of each fold, 15% of the images (stratified) were set aside as an inner validation set. It was used only for early stopping and learning rate reduction, so the test fold never influenced training decisions [26][27]. The share is set in the notebook as INNER_VAL_FRAC = 0.15. It lies between the validation shares of the base studies, 12.5% in Aminah et al. [7] and 20% in Aji et al. [8], and keeps 11 images per class for early stopping while leaving 61 per class for fitting the model weights. The training portion of each fold holds 288 images, and scikit-learn rounds the held-out share up [43], so each fold had 44 inner validation and 244 training images (calculated: 0.15 × 288 = 43.2, rounded up to 44), which matches the image counts printed by the notebook. Because the classes are balanced, each fold contained 61 training, 11 inner validation, and 18 test images per class (calculated). Table 3.2 shows the resulting sizes.

Table 3.2. Data partitioning in each cross-validation fold.

Subset	Share of data	Images per fold	Use
Training	About 68%	244	Fitting the model weights
Inner validation	About 12%	44	Early stopping and learning rate reduction only
Test fold	20%	72	Reporting results only

Because the sources do not identify individual cats, splitting was done per image. Photographs of the same cat could therefore fall in different folds, which can inflate accuracy [11]. The code contains an option for splitting by cat that can be used if such identifiers become available.

3.5 Model Architectures and Training

Each architecture was loaded with ImageNet weights [19], with its original classifier removed. The same classification head was added to all three: global average pooling, a dense layer of 256 units with ReLU activation, dropout of 0.4, and a four-class softmax output. Using one head for all models keeps the comparison about the backbones. The head structure follows the pooling, dropout, and softmax heads used in earlier feline work [8][9]; the layer size and dropout rate were design choices.

Training had two phases. In Phase 1 the backbone was frozen and only the head was trained. In Phase 2 the last 30% of each backbone's layers were unfrozen (52 of 175 layers for ResNet50, 71 of 238 for EfficientNetB0, and 46 of 154 for MobileNetV2), with batch normalization layers kept frozen, and training continued at a lower learning rate. This follows the pattern used by Aji et al., who trained the head first and then fine-tuned the last layers [8]; Syahputra and Arifudin also found that fine-tuning the upper layers improved accuracy [9].

Because the best number of layers to fine-tune differs between tasks [20], the same proportion of layers was unfrozen in each model so that no model received more fine-tuning than another; the value of 30% was not tuned.

The epoch numbers in Table 3.3 (20 for Phase 1 and 50 for Phase 2) are upper limits, not fixed training lengths. After each epoch, the loss on the inner validation set is checked. If it has not improved for 5 consecutive epochs in Phase 1, or for 10 in Phase 2, training stops and the weights from the epoch with the lowest validation loss are restored [27]. A phase that stops early after epoch e therefore reached its best validation loss at epoch e − 5 (Phase 1) or e − 10 (Phase 2); a phase that keeps improving runs to its limit. Training can therefore end well before the epoch limit: the earliest possible stop is after epoch 6 in Phase 1 and after epoch 11 in Phase 2, when the best validation loss occurs in the first epoch of the phase, and a phase whose validation loss last improves at epoch 7 stops after epoch 12 (Section 3.6.4, Table 3.4). For example, in fold 1, MobileNetV2's Phase 1 ended after epoch 18 because its validation loss last improved at epoch 13, while ResNet50 and EfficientNetB0 used all 20 Phase 1 epochs (Section 4.2). Stopping in this way prevents a model from continuing to fit the small training set after it has stopped improving on unseen images.

Table 3.3 lists all training settings and the basis for each. All settings were fixed in advance and shared by the three models; no hyperparameter search was done (Section 1.6). Training ran on Google Colab with a T4 GPU (the notebook's runtime setting), using TensorFlow 2.21.0 (the version printed by the notebook) and its Keras API (Keras 3.13.2), with scikit-learn, SciPy, statsmodels, and imagehash for data handling and evaluation. Operation-level determinism was enabled in TensorFlow, together with the random seed, to make runs reproducible. The notebook also records the versions of the other libraries: Python 3.13.15, NumPy 2.1.3, pandas 2.2.3, scikit-learn 1.6.1, SciPy 1.16.3, statsmodels 0.15.0, and CUDA 12.5.1.

Table 3.3. Training settings and their basis.

Setting	Value	Basis
Pretrained weights	ImageNet	Transfer learning evidence [8][20]
Input size	224 x 224 x 3	Input size of the ImageNet pretrained weights [7]
Classification head	Global average pooling, Dense 256 (ReLU), Dropout 0.4, Softmax (4)	Structure as in [8][9]; size and dropout were design choices
Phase 1	Backbone frozen; Adam; learning rate 0.001; up to 20 epochs	Adam [44]; 0.001 was the best learning rate in [8]
Phase 2	Last 30% of backbone layers unfrozen (batch normalization frozen); learning rate 0.0001; up to 50 epochs	Partial fine-tuning as in [8][9]; proportion was a design choice [20]
Batch size	32	Same as [8]
Loss function	Categorical cross-entropy	Multi-class loss used by [3][17]

Early stopping	Inner validation loss; patience 5 (Phase 1) and 10 (Phase 2); best weights restored	[27]; patience 10 as in [7]
Learning rate reduction	Factor 0.5 when validation loss stalls; patience 3 (Phase 1) and 5 (Phase 2); minimum 0.000001 (Phase 1) and 0.0000001 (Phase 2)	Factor 0.5 and patience 5 as in [7]
Class weights	Balanced, computed per fold (all equal to 1.00 in the final, balanced dataset)	Kept to offset class imbalance if present [29]
Augmentation	Rotation 30 degrees, shift 10%, zoom 15%, horizontal and vertical flip; training images only	Operations as in [8][41]; ranges were design choices
Random seed	42	Reproducibility

Training objective. The formulas below describe how each model turns an image into class probabilities and how it is corrected during training.

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \quad K = 4 \tag{3.2}$$

z_k is the output of the final dense layer for class k, and p_k is the predicted probability of class k. The four probabilities are positive and sum to 1, and the predicted class

is the one with the highest p_k. Softmax is the standard output function for multi-class classification [45]. In this study it produces the class probabilities from which every metric in Section 3.6 is computed.

$$w_k = \frac{N_{\text{train}}}{K \cdot N_k} \qquad (3.3)$$

N_train is the number of training images in the fold, K = 4 is the number of classes, and N_k is the number of training images of class k. This is the balanced class weight formula of scikit-learn's compute_class_weight [43], which the notebook applied separately in each fold. A class with fewer images receives a weight above 1. In the final dataset, every fold's training set held 61 images per class, so every weight equals 244 / (4 × 61) = 1.00 (calculated), and the weighted loss below reduces to ordinary categorical cross-entropy. The weighting remains in the pipeline because it would offset imbalance if class sizes changed.

$$L = -\frac{1}{B}\sum_{i=1}^{B} w_{y_i} \log p_{i,y_i} \qquad (3.4)$$

B is the number of images in a training batch (32), y_i is the true class of image i, p_i,y_i is the probability the model gives to that true class (Equation 3.2), and w_y_i is its class weight (Equation 3.3). Without the weights, this is categorical cross-entropy: the negative logarithm of the probability given to the correct class, which is small when the model is confident and correct and large when it is confident and wrong [45]. Hwang et al. and Manoj et al. trained with the same loss [3][17]. The notebook passes the weights to Keras through the class_weight argument of the training function. The notebook uses TensorFlow 2.21.0, in which tf.keras refers to Keras 3 by default from TensorFlow 2.16 onward [46]; the notebook does not switch to the older Keras 2, and it reports Keras 3.13.2. In Keras 3,

class_weight weights the loss function during training only [47]: each image's loss is multiplied by the weight of its class, and the default loss reduction, sum_over_batch_size, adds the weighted losses and divides by the number of images in the batch, not by the sum of the weights [47]. Equation 3.4 therefore matches the implementation, with B equal to the number of images in the batch: 32, except for the last batch of each epoch, which holds 20 of the 244 training images (calculated: 244 = 7 × 32 + 20). Because the weights apply only during training, the validation loss used for early stopping is unweighted.

3.6 Evaluation

3.6.1 Classification Performance

Every classification metric comes from the confusion matrix C of a test fold. C is a 4 × 4 table in which C_ij is the number of test images whose true class is i and whose predicted class is j. For each class k, the other three classes are treated together as "not k", which gives four counts:

$$TP_k = C_{kk}, \quad FP_k = \sum_{i \neq k} C_{ik}, \quad FN_k = \sum_{j \neq k} C_{kj}, \quad TN_k = N - TP_k - FP_k - FN_k \qquad (3.5)$$

TP_k (true positives) are images of class k predicted as k. FP_k (false positives) are images of other classes predicted as k. FN_k (false negatives) are images of class k predicted as another class. TN_k (true negatives) are all remaining images. N is the number of test images (72 per fold). Breaking a multi-class result into these one-versus-rest counts is the approach described by Sokolova and Lapalme [30]. The confusion matrix itself is reported (Figures 4.5a to 4.5c) because it shows which pairs of classes are confused, which tests the expectation in Section 2.1 that flea allergy dermatitis and ringworm are the pair most likely to be mixed up, and shows how often each disease is mistaken for healthy skin.

$$\text{Accuracy} = \frac{\sum_{k=1}^{K} C_{kk}}{N} \qquad (3.6)$$

Accuracy is the share of all test images classified correctly. It answers the overall part of RO1, and it is the measure reported by every feline study reviewed [7][8][9], so it allows comparison with them. Because it combines all classes, it can stay high while one class is handled poorly [28], so it is always reported with the class level measures below. Equation 3.6 is the multi-class form of accuracy, computed directly from the confusion matrix.

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k}, \qquad \text{Recall}_k = \frac{TP_k}{TP_k + FN_k}, \qquad F1_k = \frac{2\ \text{Precision}_k\ \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k} \qquad (3.7)$$

Precision_k tells how often a prediction of class k is correct. Recall_k, also called sensitivity, tells what share of the images of class k were found. F1_k is the harmonic mean of the two and is high only when both are high [30]. These are needed because the three diseases overlap visually (Section 2.1): a model can reach good accuracy while still missing many ringworm images, which only recall reveals. Aji et al. and Syahputra and Arifudin reported the same per class measures for feline skin classes [8][9], and Hwang et al. reported sensitivity for dog skin models [3].

$$\text{Specificity}_k = \frac{TN_k}{TN_k + FP_k} \qquad (3.8)$$

Specificity_k is the share of images that do not belong to class k and were correctly not assigned to it [30]. Low specificity means a class attracts wrong predictions from other classes; for the healthy skin class, it means that diseased images are being predicted as

healthy, so healthy-class specificity equals the screening sensitivity defined below. Hwang et al. reported specificity for each dog skin model for this reason [3].

Disease-versus-healthy screening. To measure the error that matters most for screening, the notebook also collapses the four classes into two, diseased (any of the three diseases) and healthy. Screening sensitivity is the share of diseased test images that were not predicted as healthy, and screening specificity is the share of healthy test images that were predicted as healthy. A diseased image predicted as the wrong disease still counts as detected in this view, because it would still be flagged for veterinary attention.

$$\text{Macro-F1} = \frac{1}{K}\sum_{k=1}^{K} F1_k \qquad (3.9)$$

Macro-F1 averages the per class F1-scores with equal weight, so every class counts equally regardless of its size. This is the version computed by scikit-learn with average="macro" [43]. Sokolova and Lapalme also describe a variant that computes F1 from macro-averaged precision and recall [30]; that variant was not used. Macro-averaged precision and recall are computed in the same way, by averaging the per class values. Macro-F1 is reported for every fold because it summarizes RO1 at the class level in one number.

$$\text{MCC} = \frac{c \cdot N - \sum_{k=1}^{K} p_k \, t_k}{\sqrt{(N^2 - \sum_{k=1}^{K} p_k^2)(N^2 - \sum_{k=1}^{K} t_k^2)}} \qquad (3.10)$$

Here c = Σ C_kk is the number of correct predictions, t_k is the number of images whose true class is k, and q_k is the number of images predicted as k (written q_k to distinguish it from the predicted probability p_k in Equation 3.2). MCC ranges from −1 to

+1: +1 means perfect prediction, 0 means no better than chance, and negative values mean systematic disagreement. This is the multi-class form of the Matthews correlation coefficient proposed by Gorodkin [31] and implemented in scikit-learn [43]. MCC is used because it is high only when all classes are predicted well and it accounts for class sizes [28]; Hwang et al. used MCC for the same reason [3]. It gives RO1 a single summary that is harder to inflate than accuracy.

Worked example (ResNet50, fold 1; Figure 4.5a). Of the 18 ringworm test images, 12 were predicted as ringworm, and two images of other classes were predicted as ringworm (one flea allergy dermatitis and one scabies image), so ringworm precision = 12 / (12 + 2) = 0.857 and recall = 12 / 18 = 0.667. For flea allergy dermatitis, TP = 15, FN = 3, FP = 4 (three from ringworm and one from scabies), and TN = 72 − 15 − 3 − 4 = 50, so precision = 15 / 19 = 0.789 and specificity = 50 / 54 = 0.926. These values match the fold-1 classification report printed by the notebook; Tables 4.2 and 4.2a report the same metrics pooled over all 360 out-of-fold predictions.

$$\bar{x} = \frac{1}{F}\sum_{f=1}^{F} x_f, \qquad s = \sqrt{\frac{1}{F}\sum_{f=1}^{F}(x_f - \bar{x})^2}, \quad F = 5 \qquad (3.11)$$

x_f is a metric (accuracy, macro-F1, or MCC) on test fold f. The mean summarizes performance, and s shows how much it changes between folds. The notebook computed s with NumPy's std function at its default setting (ddof = 0), which divides by the number of values F, the population form [48]. Dividing by F − 1 (ddof = 1) would give slightly larger values; for example, the ResNet50 accuracy standard deviation would be 4.61 instead of 4.12 (calculated from the five fold accuracies).

$$f^* = \operatorname{argmin}_f \left| \bar{a}_f - \bar{a} \right| \qquad (3.12)$$

Per class metrics, specificity, and the screening measure are reported on the 360 pooled out-of-fold predictions, in which each image is predicted once, by the model of the fold in which it was tested. Training curves, the confusion-matrix figures, and the Grad-CAM examples are shown for one representative fold. ā_f is the accuracy on fold f averaged over the three models, and ā is the average over all folds; the representative fold f* is the one closest to the overall average. This rule was a design choice applied in the notebook; if several folds are equally close to the overall average, the notebook's argmin function returns the first of them, so ties are broken by fold order. The fold selected is reported in Section 4.2. Because the representative fold is used only for these illustrations, its selection does not affect the reported per class results.

3.6.2 Statistical Comparison of Classification Accuracy

$$\mathrm{CI}_{95\%} = \left[\hat{\theta}^*_{(2.5)}, \hat{\theta}^*_{(97.5)} \right] \qquad (3.13)$$

RO2 calls for a descriptive comparison and an inferential one; the full comparison procedure is set out in Section 3.7. This section defines the statistical methods used in it: bootstrap confidence intervals, which show the uncertainty of each architecture's observed result, and McNemar's test, which asks whether the observed differences in classification accuracy are larger than would be expected by chance. Because the three architectures are tested on the same images in the same folds, their predictions are paired, and tests that treat their accuracies as independent samples are not appropriate [32].

To show the uncertainty of each model's overall result, the 360 out-of-fold predictions of a model (each image predicted once, in the fold where it was tested) were pooled. B = 2,000 bootstrap samples were drawn, each by selecting 360 of those predictions at random with replacement, and the metric θ*_b (accuracy, macro-F1, or MCC, all computed on the same resamples) was computed on each sample b. The 2.5th and 97.5th percentiles of the 2,000 values form the 95% confidence interval. This is the percentile bootstrap interval described by Efron and Tibshirani [49]. It is needed because every estimate rests on only 360 images; the interval shows how much the estimate could change with a different sample of images. The random generator was seeded with the study seed (42).

To test whether two models differ, McNemar's test was used. It was applied to each of the three pairs: ResNet50 and EfficientNetB0, ResNet50 and MobileNetV2, and EfficientNetB0 and MobileNetV2. For each pair, the null hypothesis H0 is that the two architectures have the same error rate on the test images, and the two-sided alternative H1 is that their error rates differ. For models A and B tested on the same n images, n01 is the number of images that A classified correctly and B incorrectly, and n10 the reverse. Images that both models got right, or both got wrong, do not affect the test. If the two models have the same error rate, n01 and n10 are equally likely.

$$p = \min\left(1,\ 2\sum_{i=0}^{\min(n_{01},\,n_{10})}\binom{n}{i}0.5^{n}\right), \quad n = n_{01} + n_{10} \qquad (3.14)$$

$$\chi^2 = \frac{(|n_{01} - n_{10}| - 1)^2}{n_{01} + n_{10}}, \quad \text{1 degree of freedom} \qquad (3.15)$$

The notebook calls the mcnemar function of statsmodels (version 0.15.0), requesting the exact binomial test (Equation 3.14) when n is below 25 and the continuity-corrected

chi-square form (Equation 3.15) otherwise. For example, n01 = 7 and n10 = 3 (n = 10, below 25) give p = 2 × (1 + 10 + 45 + 120) / 2^{10} = 0.344, the value printed by the notebook (0.3438). The significance level was 0.05. Dietterich recommends McNemar's test, in the form of Equation 3.15, for comparing two classifiers that are each trained once and tested on the same data, because its false positive rate is acceptable in that setting [32]. The test answers the inferential part of RQ2 and RO2. For RQ2, the test was applied to all 360 pooled out-of-fold predictions. Every image has exactly one prediction from each architecture, so the predictions are paired by image; the notebook checked that the three architectures had the same images, in the same order, with the same fold assignments and true labels. The same rule selects the form of the test for each pair: the continuity-corrected chi-square form (Equation 3.15) when n01 + n10 is at least 25, and the exact binomial test otherwise; the form used for each pair is reported with the results (Table 4.4). Because three pairwise tests were made, the p-values were adjusted with the Holm step-down procedure [54], and significance was judged on the adjusted p-value at α = 0.05. The same test applied to the 72 images of fold 1 is reported only as an exploratory analysis (Section 4.4.2). One limitation remains: pooled predictions come from five separately trained models per architecture, and McNemar's test does not account for this training variation [32].

Equation 3.14 also gives a useful bound. When two models differ by d correctly classified images, the most extreme split is n01 = d and n10 = 0, which gives p = 2 × 0.5^d. Section 4.4.2 applies this bound to the observed difference between ResNet50 and EfficientNetB0. Overlapping confidence intervals were not treated as proof of no difference [33], and a non-significant result was not treated as proof that two models are equal [34].

3.6.3 Computational Characteristics

Four computational characteristics were recorded for each architecture to address the computational part of RO1: number of parameters, FLOPs, model file size, and inference time. Parameters and FLOPs are hardware-independent properties of the architecture, while model file size and inference time depend on the software and hardware environment in which they were measured. The total number of parameters and its split into trainable and non-trainable parameters were recorded from Keras' weight lists for both training phases. In Phase 1, when the backbone was frozen, the trainable parameters were those of the classification head: 525,572 for ResNet50 and 328,964 for MobileNetV2 and EfficientNetB0 (also calculated below). Phase 2 is the state of the final evaluated models, in which the unfrozen backbone layers are also trainable; the notebook confirmed that the saved models of all five folds were in this state.

$$P_{\text{conv}} = k^2\, d_i\, d_j\ \left(+\, d_j \text{ if the layer has a bias}\right), \qquad P_{\text{dense}} = n_{\text{in}}\, n_{\text{out}} + n_{\text{out}} \qquad (3.16)$$

k is the kernel size, and d_i and d_j are the numbers of input and output channels; a convolution kernel holds k × k × d_i × d_j weights [23]. A dense layer with n_in inputs and n_out outputs holds n_in × n_out weights and n_out biases. Batch normalization layers add a small number of values per channel. Keras counts all of these. Worked example: the shared head adds 2,048 × 256 + 256 = 524,544 and 256 × 4 + 4 = 1,028 parameters to ResNet50 (525,572 in total), and 1,280 × 256 + 256 + 1,028 = 328,964 parameters to MobileNetV2 and EfficientNetB0, whose last feature layers have 1,280 channels. Subtracting these from the totals in Table 4.3 gives backbones of 23,587,712, 4,049,571, and 2,257,984 parameters for ResNet50, EfficientNetB0, and MobileNetV2 (calculated). Parameters matter for RO1 and RO3 for two reasons: they determine how much memory the model needs, and a model with more parameters must learn them from only 244 training images per fold, which bears on its classification performance.

$$\mathrm{MAC}_{\mathrm{conv}} = h_o\, w_o\, k^2\, d_i\, d_j, \qquad \mathrm{FLOPs} \approx 2 \times \mathrm{MACs} \qquad (3.17)$$

h_o × w_o is the size of the layer's output feature map. A multiply-accumulate (MAC) is one multiplication followed by one addition; the cost of a standard convolution in MACs follows Sandler et al. [23]. The TensorFlow profiler used in the notebook counts floating point operations (FLOPs) and counts the multiplication and the addition separately, so FLOPs are about twice the MACs. The architecture papers report MACs (called multiply-adds): about 3.8 billion for ResNet50 [22], 300 million for MobileNetV2 [23], and 0.39 billion for EfficientNetB0 [24]. Doubling these gives about 7.6, 0.60, and 0.78 billion, close to the measured 7.75, 0.61, and 0.80 GFLOPs (Table 4.3); the small differences are most likely due to rounding in the published figures and to the profiler counting operations that the papers did not count, such as batch normalization, activations, additions, and pooling (the new classification heads add only about 0.001 GFLOPs). This check shows that the measurements agree with the published designs. FLOPs are reported because they measure the computation needed for one prediction without depending on hardware [35].

$$\bar{t} = \frac{1}{R}\sum_{r=1}^{R} t_r\,, \quad R = 50 \qquad (3.18)$$

t_r is the time of the r-th single-image prediction (batch size 1), measured after 5 warm-up predictions on the T4 GPU; the standard deviation uses the form in Equation 3.11. The input was a random 224 × 224 × 3 array, since the pixel values do not change the amount of computation. Inference time is included because FLOPs do not fully predict speed, which also depends on memory access and on the hardware [35]; Aminah et al. measured CPU inference time for the same reason [7]. All three models were timed in the same session. Model size is the size of the saved .keras file in megabytes (bytes divided by 1,024²). The file

saved by the training procedure includes the Adam optimizer state, so the same model and weights were also saved without optimizer state, in the same format; this inference-only size is the one relevant to using a trained model.

3.6.4 Asymptotic Computational Complexity

The measured values in Section 3.6.3 describe the three trained models at one input size. Asymptotic analysis explains why their computational characteristics differ and how they grow with input size. This supports the interpretation of the computational results (RO1) and of the trade-offs between architectures (RO3). It describes how the number of operations (time complexity) or stored values (space complexity) grows as the input grows, ignoring constant factors [50].

$$f(n) = O(g(n)) \Leftrightarrow \exists\, c > 0,\ n_0 > 0 \colon 0 \le f(n) \le c\, g(n) \ \text{ for all } n \ge n_0 \qquad (3.19)$$

O(g(n)) is an upper bound on the growth of f(n), and Θ(g(n)) means that f(n) grows at the same rate as g(n), within constant factors, from above and below [50]. The three architectures differ mainly in constant factors, so the asymptotic results below are read together with the measured FLOPs.

Standard convolution (the building block of ResNet50). For one layer with an h × w output, kernel size k, d_i input channels, and d_j output channels:

$$T_{\text{std}} = \Theta\left(h\, w\, k^2\, d_i\, d_j\right), \qquad S_{\text{std}} = \Theta\left(k^2\, d_i\, d_j\right) \qquad (3.20)$$

T is time (operations) and S is space (parameters). Time grows with the image area h × w, the kernel area k², and the product of input and output channels [23].

Depthwise separable convolution (used in MobileNetV2 and EfficientNetB0). The depthwise step filters each input channel separately, and a 1 × 1 pointwise step then combines channels [23]:

$$T_{\mathrm{dws}} = \Theta\left(h\, w\, d_i\, (k^2 + d_j)\right), \qquad \frac{T_{\mathrm{dws}}}{T_{\mathrm{std}}} = \frac{1}{d_j} + \frac{1}{k^2} \qquad (3.21)$$

The ratio follows from dividing the two costs. With k = 3 and many output channels, it approaches 1/9, which is the "8 to 9 times" reduction reported by Sandler et al. for MobileNetV2 [23]. This relation, together with differences in channel width, contributes substantially to the gap in FLOPs between ResNet50 and the two lightweight models in Table 4.3.

Inverted residual block (MobileNetV2; the MBConv blocks of EfficientNetB0 are of the same type [24]). For an h × w block with expansion factor t, kernel size k, d′ input channels, and d″ output channels, Sandler et al. give the number of multiply-adds as [23]:

$$T_{\mathrm{inv}} = h \cdot w \cdot d' \cdot t\,(d' + k^2 + d'') \qquad (3.22)$$

The block expands the input to t × d′ channels, filters them depthwise, and projects them back to d″ channels. Because d′ and d″ are kept small, the block stays cheap even though it works internally with more channels.

Residual connection (ResNet50). A residual block adds its input x to its output F(x) [22]:

$$y = \mathcal{F}(x, \{W_i\}) + x \qquad (3.23)$$

The addition costs Θ(h × w × d) operations and adds no parameters [22], so residual connections make very deep networks trainable without changing their order of cost.

Whole network and input size. Summing Equation 3.20, 3.21, or 3.22 over all layers gives the cost of one prediction. For a fixed architecture, each layer's feature map area grows with the input area, so for an input of side r:

$$T_{\text{net}}(r) = \Theta(r^2) \text{ per image}, \qquad T_{\text{net}}(n) = \Theta(n) \text{ for } n \text{ images} \qquad (3.24)$$

Tan and Le state the same relation for the whole network: FLOPs grow in proportion to depth d, to the square of width w, and to the square of resolution r [24]. Their compound scaling rule, which defines the EfficientNet family, is:

$$d = \alpha^{\phi}, \quad w = \beta^{\phi}, \quad r = \gamma^{\phi}, \qquad \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2 \qquad (3.25)$$

Each step of the coefficient φ roughly doubles FLOPs, and EfficientNetB0 is the base model with φ = 0 [24]. For space, the number of parameters does not depend on the input size (Equations 3.16 and 3.20), while the memory for intermediate feature maps grows with the image area, $\Theta(r^2)$.

Interpretation for this study. All three models have the same order of growth: $\Theta(r^2)$ per image and Θ(n) for n images. Asymptotic order alone therefore cannot separate them. They differ in the constant factor, the cost per pixel, which is set by their layer designs (Equations 3.20 to 3.22). This is why the study reports measured FLOPs in addition to the asymptotic analysis. The analysis also shows that using a larger input than 224 × 224 would raise the cost of all three models quadratically. This matters for the research problem because a screening tool would need to run on ordinary devices (Section 1.1).

Best-case and worst-case complexity of the study's procedures. For a given input size, the worst-case running time of an algorithm is its longest running time over all inputs of that size, and the best-case running time is its shortest; $O(g(n))$ gives an upper bound, $\Omega(g(n))$ a lower bound, and $\Theta(g(n))$ both [50]. When the best and worst cases have the same order, the running time is Θ of that order for every input. Table 3.4 applies this analysis to the main procedures in the notebook, with n images, $N = 360$ pooled predictions, $K = 4$ classes, $B = 2{,}000$ bootstrap samples, $m = 3$ pairwise tests, F the FLOPs of one prediction (Table 4.3), $N_{train} = 244$ training images per fold, and c_1 and c_2 the cost of processing one training image in a Phase 1 and a Phase 2 epoch.

Table 3.4. Best-case and worst-case time complexity of the study's procedures.

Procedure	**Best case**	**Worst case**	**In this study**
Perceptual hashing of n images	$\Theta(n)$	$\Theta(n)$	360 hashes; every image has the same size, so each hash costs O(1)
Pairwise comparison of hashes	$\Theta(n^2)$	$\Theta(n^2)$	64,620 comparisons; every pair is always compared
Grouping near-duplicates (union-find with path halving)	$\Theta(n)$: no pair within the threshold, so no merges	$O(n^2 \log n)$: every pair within the threshold, $n(n-1)/2$ merges at $O(\log n)$ amortized each [50]	Best case: no pair had $d_H \leq 5$ (Section 3.2)

Training one model on one fold (early stopping)	17 epochs (6 + 11): $\Theta(N_train(6c_1 + 11c_2))$	70 epochs (20 + 50): $\Theta(N_train(20c_1 + 50c_2))$	Fold 1: 39 (ResNet50), 48 (EfficientNetB0), and 39 (MobileNetV2) epochs (Section 4.2)
Cross-validation (3 architectures × 5 folds)	15 × 17 = 255 model-epochs	15 × 70 = 1,050 model-epochs	74.1 minutes on the T4 GPU
Inference on one image	$\Theta(F)$	$\Theta(F)$	F = 7.75, 0.80, and 0.61 GFLOPs, the same for every image
Metrics from N predictions (confusion matrix, accuracy, macro-F1, MCC)	$\Theta(N + K^2)$	$\Theta(N + K^2)$	N = 360, K = 4
Bootstrap confidence intervals	$\Theta(B(N + K^2))$	$\Theta(B(N + K^2))$	B = 2,000 resamples per model
McNemar tests and Holm correction	$\Theta(N + m \log m)$	$\Theta(N + m \log m)$	m = 3 pairwise tests
Grad-CAM for one	$\Theta(F)$	$\Theta(F)$	One forward and one

image			backward pass

Three results follow from Table 3.4. First, most procedures have the same best and worst case, because they perform the same operations whatever the data: deduplication always compares all 64,620 pairs, and each CNN performs the same operations on every image, whatever its content. The variation in measured inference time (Section 4.3.3) therefore comes from the software and hardware environment, not from the models themselves. Second, the grouping step of deduplication has different best and worst cases, and this study was in the best case because no pair of images had a Hamming distance of 5 or less (Section 3.2). Third, early stopping makes the training cost depend on the data: each model trains for between 17 and 70 epochs on each fold, so the cross-validation as a whole lies between 255 and 1,050 model-epochs, and in fold 1 the three models trained for 39, 48, and 39 epochs (Section 4.2).

3.6.5 Error Analysis

$$\alpha_k^c = \frac{1}{Z}\sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k}, \qquad L^c_{\text{Grad-CAM}} = \text{ReLU}\left(\sum_k \alpha_k^c A^k\right) \qquad (3.26)$$

A^k is the k-th feature map of the last convolutional layer, A^k_ij is its value at position (i, j), Z is the number of positions, and y^c is the model output for class c. α_k^c measures how much feature map k supports class c, and the weighted sum, with negative values removed by ReLU, is a heatmap of the regions that increased the output for class c [36]. Selvaraju et al. take the gradient of the class score before softmax; the notebook took it of the softmax probability of the predicted class and scaled each heatmap to a maximum of 1. Grad-CAM was applied to one test image of each class in the representative fold (the first image of each class) for each model. With only four images per model, all correctly

classified, this analysis is illustrative and does not show how the models behave in general. It is used only to inform the discussion of possible weaknesses under RO3, such as reliance on photograph framing, and is not a performance metric.

3.7 Comparison of the Classification Models

This section describes how the three architectures are compared to answer RQ2 and RO2, and how the comparison is used to answer RQ3 and RO3. It brings together the classification metrics defined in Section 3.6.1, the computational characteristics defined in Section 3.6.3, and the statistical methods defined in Section 3.6.2 into a single comparison procedure. The results of the comparison are reported in Section 4.4, and the strengths and weaknesses identified from it in Section 4.5.

3.7.1 Comparison Conditions

The comparison is a controlled, paired comparison. The three architectures are trained and tested under identical conditions (Section 3.1): the same 360 images, the same five stratified folds (random seed 42), the same preprocessing and training-only augmentation, the same classification head, and the same two-phase training schedule and hyperparameters, with no architecture-specific tuning (Sections 3.3 to 3.5). In every fold, each architecture is tested on the same 72 held-out images, so all three produce exactly one out-of-fold prediction for each of the 360 images, and their predictions are paired by image. Only the held-out test folds are used to compare classification performance; the inner validation set is used only for early stopping and learning rate reduction (Section 3.4), because it influences training decisions. Computational characteristics are measured for all three architectures in the same Google Colab session on the same T4 GPU (Section 3.6.3).

3.7.2 Comparison Steps and Metrics

The comparison proceeds in five steps.

Step 1, overall classification performance. Accuracy, macro-F1, and MCC are computed on each test fold and summarized as the mean and standard deviation over the five folds, and on the 360 pooled out-of-fold predictions with 95% percentile bootstrap confidence intervals (Sections 3.6.1 and 3.6.2). The architectures are ranked on each metric, and their per-fold accuracies are compared to show whether the ranking holds in every fold.

Step 2, per-class and screening performance. Per-class precision, recall, F1-score, and specificity, and the disease-versus-healthy screening sensitivity and specificity, are computed on the 360 pooled out-of-fold predictions and compared between architectures; the confusion matrices of the representative fold show which pairs of classes each architecture confuses (Section 3.6.1). These differences are compared descriptively only and are not tested for significance, because with 90 test images per class a difference of a few images changes a per-class value noticeably.

Step 3, computational characteristics. The number of parameters, FLOPs, model file size (with and without optimizer state), and single-image inference time are compared, and the differences in parameters and FLOPs are expressed as ratios between architectures (Section 3.6.3). In addition to the eager-execution timing, a supplementary graph-mode timing (tf.function, 20 warm-up and 3 × 200 timed passes per model) is run so that framework overhead can be separated from computation. Parameters, FLOPs, and model size depend only on the architecture and head, not on the fold, so they are not tested statistically.

Step 4, statistical test of accuracy differences. Each of the three pairs of architectures is compared with McNemar's test on the 360 pooled, paired out-of-fold predictions, the p-values are adjusted with the Holm procedure, and significance is judged at $\alpha = 0.05$ (Section 3.6.2). The same test on fold 1 is reported as an exploratory analysis only.

Step 5, joint assessment. The results of Steps 1 to 4 are combined for each architecture to identify its strengths and weaknesses, using the criteria in Section 3.7.3.

3.7.3 Criteria for Judging Performance and Identifying Strengths and Weaknesses

The following criteria are used both to judge the differences found in the comparison (Section 4.4) and to identify the strengths and weaknesses of each architecture for RQ3 and RO3 (Section 4.5). Each architecture is assessed on four aspects: classification strengths, classification weaknesses, computational strengths, and computational weaknesses. Its practical implications are then stated. The assessment uses only the results reported in Sections 4.3 and 4.4, and it follows three principles. First, a difference in classification accuracy is described as a strength or weakness only with its statistical status: an observed difference that was not shown to be significant is reported as observed, not as superiority. Second, computational characteristics are judged mainly by parameters and FLOPs, which describe the architecture independently of hardware [35]. Model file size and inference time are reported as measured in the study environment, with their limitations (Section 4.3.3). Third, no composite score is computed and no single architecture is declared best overall, because the relative importance of accuracy and computational demand depends on the intended use. The assessment therefore states the trade-offs that a user would weigh. These principles are a design decision of the study and are not taken from the literature.

Chapter 4

RESULTS AND DISCUSSION

4.1 Dataset Summary

The final dataset contained 360 images, 90 in each of the four classes (flea allergy dermatitis, healthy skin, ringworm, and scabies). The perceptual-hash search found no near-duplicate groups and no cross-class conflicts, and the notebook found no file-name collisions, so all 360 images were kept. Five-fold stratified cross-validation with a three-way per-fold split produced 244 training, 44 inner-validation, and 72 test images per fold, with every class equally represented in each subset (61, 11, and 18 images per class). No offline augmentation was performed; training-only online augmentation was applied within each fold (Section 3.3). The complete cross-validation, 15 model trainings, took 74.1 minutes.

4.2 Training Performance

All three models were trained with the same two-phase transfer-learning procedure. In Phase 1, the pretrained base was fully frozen and only the classification head was trained. In Phase 2, the last 30% of each base's layers (52 of 175 for ResNet50, 71 of 238 for EfficientNetB0, and 46 of 154 for MobileNetV2) were unfrozen, with the batch-normalization layers among them kept frozen (16, 15, and 16 layers, respectively), and fine-tuned at a learning rate of 0.0001. Figures 4.1 to 4.3 show the representative fold, fold 1, selected by the rule in Section 3.6.1: folds 1, 2, and 4 had the same three-model mean test accuracy (82.87%, or 179 of 216 correct predictions; calculated), equally close to the overall mean of 84.17% (Table 4.1a), and the tie was broken by fold order. ResNet50 and EfficientNetB0 used all 20 Phase 1 epochs, so their Phase 2 began at epoch 21.

MobileNetV2's Phase 1 stopped early after 18 epochs because its inner-validation loss had not improved for five epochs, so its Phase 2 began at epoch 19. Early stopping then ended Phase 2 after 19 epochs for ResNet50 (39 in total), 28 for EfficientNetB0 (48 in total), and 21 for MobileNetV2 (39 in total), in each case ten epochs after the lowest inner-validation loss, whose weights were restored (Section 3.5).

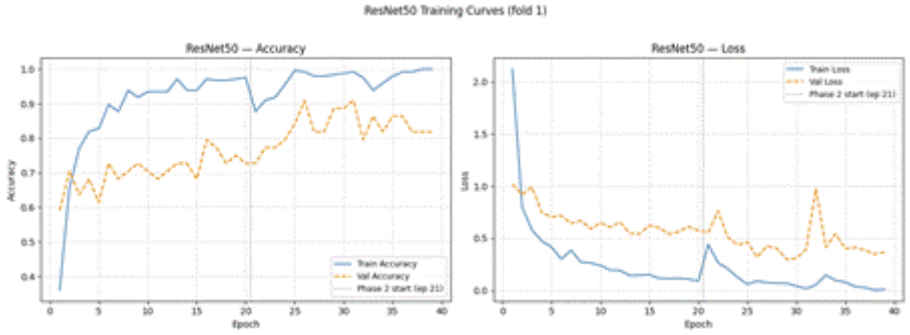


Figure 4.1. ResNet50 training curves (accuracy and loss) across Phase 1 and Phase 2, representative fold 1. Dotted line marks Phase 2 start (epoch 21).



Figure 4.2. MobileNetV2 training curves (accuracy and loss) across Phase 1 and Phase 2, representative fold 1. Dotted line marks Phase 2 start (epoch 19).

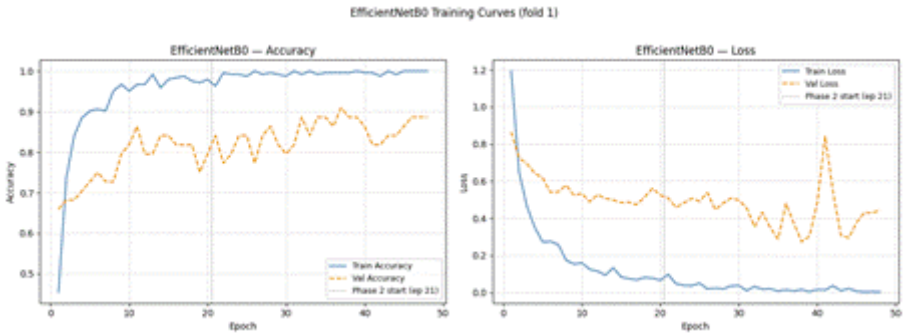


Figure 4.3. EfficientNetB0 training curves (accuracy and loss) across Phase 1 and Phase 2, representative fold 1. Dotted line marks Phase 2 start (epoch 21).

In Phase 1, ResNet50 reached about 97% training accuracy and about 70 to 80% inner-validation accuracy (Figure 4.1), MobileNetV2 about 92 to 97% and 70 to 80% (Figure 4.2), and EfficientNetB0 about 97 to 99% and 80 to 86% (Figure 4.3). At the start of Phase 2, ResNet50 and MobileNetV2 showed a temporary drop in training accuracy and a rise in loss as the newly unfrozen layers began to change. The effect was strongest for MobileNetV2, whose training accuracy fell from about 94% to 75% and whose inner-validation loss rose from about 0.57 to 0.79 at epoch 19. EfficientNetB0's transition was smooth. By the end of training, all three models reached close to 100% training accuracy, while inner-validation accuracy peaked at about 91% for all three. This persistent gap between training and validation performance indicates overfitting to the small training set (244 images per fold); early stopping with restoration of the best weights limited it but did not remove it.

Why the validation curves rise and fall sharply. The inner-validation curves contain several isolated peaks, for example the validation-loss spikes at about epoch 32 for ResNet50 (to about 0.98), epoch 41 for EfficientNetB0 (to about 0.84), and epoch 26 for MobileNetV2 (to about 0.96). Three properties of the training setup explain them. First, the inner-validation set holds only 44 images, so a single image changes validation accuracy by about 2.3 percentage points (1/44), and a few changed predictions produce visible jumps. Second,

cross-entropy loss grows sharply when the model gives a very low probability to the correct class, so two or three confidently wrong images can raise the average loss even when accuracy hardly changes. Third, in Phase 2 the unfrozen backbone layers are updated together with the head, so one epoch can change the learned features more than in Phase 1; this is why the validation curves are more unsettled in Phase 2, including the jump at the start of Phase 2 for MobileNetV2, although the largest spikes occurred later in Phase 2 (epochs 32, 41, and 26). The peaks are therefore expected noise from a small validation set rather than signs of an error in training, and they do not affect the reported models, because early stopping keeps the weights from the epoch with the lowest validation loss, not from the epoch at which training ends.

Inner-validation accuracy also overstated test accuracy. In fold 1, all three models peaked at about 91% on the inner-validation set but reached 84.72% (ResNet50 and EfficientNetB0) and 79.17% (MobileNetV2) on the held-out test fold. This is why only the test folds are used to report and compare performance (Sections 4.3 and 4.4).

4.3 Objective 1: Classification Performance and Computational Characteristics

RO1 is to evaluate the classification performance of the three architectures across the four classes and their computational characteristics. Section 4.3.1 reports overall classification performance (accuracy, macro-F1, and MCC), Section 4.3.2 the per-class metrics and disease-versus-healthy screening measures, and Section 4.3.3 the computational characteristics (number of parameters, FLOPs, model file size, and inference time). Section 4.3.4 summarizes the answer to RQ1 and RO1.

4.3.1 Overall Classification Performance

This section reports the overall classification performance of the three architectures (RQ1 and RO1). Table 4.1 reports mean ± standard deviation across the five held-out test folds. ResNet50 achieved the highest mean accuracy (86.11%), followed by EfficientNetB0 (84.72%) and MobileNetV2 (81.67%); the same ordering holds for MCC (0.820, 0.801, and 0.761) and macro-F1 (0.858, 0.846, and 0.815). MobileNetV2 showed the largest fold-to-fold variability (±5.08 percentage points, population standard deviation; fold accuracies from 75.00% to 90.28%), followed by ResNet50 (±4.12; 80.56% to 93.06%) and EfficientNetB0 (±3.83; 80.56% to 91.67%). Figure 4.4 shows the means and standard deviations, and Table 4.1a gives the accuracy of every fold.

Table 4.1. Test-set classification performance (five-fold cross-validation).

Model	Accuracy (%) mean ± SD	Macro-F1 mean ± SD	MCC mean ± SD	Pooled accuracy (%) [95% CI]	Pooled macro-F1 [95% CI]	Pooled MCC [95% CI]
ResNet50	86.11 ± 4.12	0.858 ± 0.044	0.820 ± 0.053	86.11 [82.5, 89.7]	0.862 [0.826, 0.898]	0.815 [0.767, 0.864]
EfficientNet B0	84.72 ± 3.83	0.846 ± 0.039	0.801 ± 0.049	84.72 [81.1, 88.3]	0.848 [0.810, 0.883]	0.796 [0.748, 0.845]

MobileNet V2	81.67 ± 5.08	0.815 ± 0.052	0.761 ± 0.067	81.67 [77.5, 85.6]	0.817 [0.774, 0.856]	0.756 [0.702, 0.810]

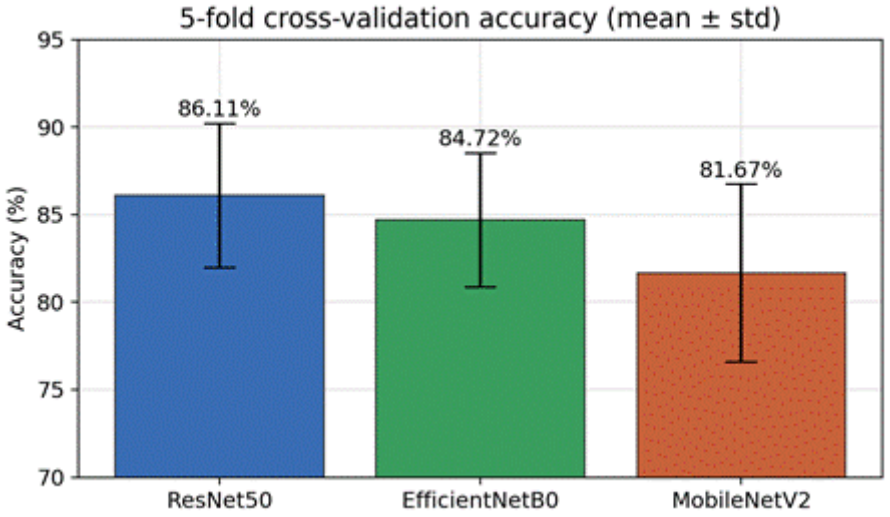


Figure 4.4. Cross-validation accuracy (mean ± std) for the three architectures.

Table 4.1a. Per-fold test accuracy (%) by architecture.

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
ResNet50	84.72	87.50	93.06	80.56	84.72
EfficientNetB0	84.72	80.56	91.67	84.72	81.94
MobileNetV2	79.17	80.56	90.28	83.33	75.00
Mean of three models	82.87	82.87	91.67	82.87	80.56

Fold 3 was the easiest fold for all three models (90.28% to 93.06%) and fold 5 the hardest on average, so the level of accuracy depends partly on which images fall in the test fold. The ranking of the architectures across folds and the significance of their differences are examined under RO2 (Section 4.4).

4.3.2 Per-Class Classification Performance and Disease-versus-Healthy Screening

This section reports the per-class and disease-versus-healthy screening part of RQ1 and RO1. Per-class results pooled over all 360 out-of-fold predictions (90 images per class) are shown in Table 4.2, with specificity and the disease-versus-healthy screening results in Table 4.2a. Ringworm was the hardest class for every model. It had each model's lowest F1-score (0.798 for ResNet50, 0.754 for EfficientNetB0, and 0.735 for MobileNetV2), lowest precision (0.716 to 0.765), and lowest specificity (0.900 to 0.915). Ringworm recall differed between models: 0.833 for ResNet50, 0.767 for EfficientNetB0, and 0.756 for MobileNetV2. Scabies had the highest F1-score for ResNet50 (0.901) and EfficientNetB0 (0.891), and healthy skin for MobileNetV2 (0.882). Healthy skin recall ranged from 0.878 (ResNet50) to 0.911 (MobileNetV2), and flea allergy dermatitis lay between the extremes (F1 0.811 to 0.862). This pattern is consistent with the visual overlap of hair loss and scaling between ringworm and flea allergy dermatitis (Section 2.1).

Two cautions apply to these per-class results. First, each class has 90 test images, so a difference of a few images changes a per-class value noticeably, and per-class differences between models were not tested for significance. Second, Figures 4.5a to 4.5c, and the error counts below, come from the representative fold (fold 1, n = 72; selection in Section 4.2). These figures illustrate the error pattern but are not the reported per-class results; Figure 4.5 shows the pooled per-class F1-scores of Table 4.2. Specificity was 0.900 or higher for every

class and model (Table 4.2a); the lowest values were for ringworm, which means other classes were most often predicted as ringworm.

Table 4.2. Per-class precision, recall, and F1-score (pooled out-of-fold predictions, n = 360, 90 images per class).

Model	Class	Precision	Recall	F1
ResNet50	Flea Allergy Dermatitis	0.881	0.822	0.851
	Ringworm	0.765	0.833	0.798
	Scabies	0.891	0.911	0.901
	Healthy Skin	0.919	0.878	0.898
EfficientNetB0	Flea Allergy Dermatitis	0.857	0.867	0.862
	Ringworm	0.742	0.767	0.754
	Scabies	0.918	0.867	0.891
	Healthy Skin	0.879	0.889	0.884

MobileNetV2	Flea Allergy Dermatitis	0.811	0.811	0.811
	Ringworm	0.716	0.756	0.735
	Scabies	0.899	0.789	0.840
	Healthy Skin	0.854	0.911	0.882

Table 4.2a. Per-class specificity and disease-versus-healthy screening (pooled out-of-fold predictions, n = 360).

Metric	ResNet50	EfficientNetB0	MobileNetV2
Specificity, flea allergy dermatitis	0.963	0.952	0.937
Specificity, ringworm	0.915	0.911	0.900
Specificity, scabies	0.963	0.974	0.970
Specificity, healthy skin	0.974	0.959	0.948
Screening sensitivity (diseased images not predicted as healthy)	0.974 (263/270)	0.959 (259/270)	0.948 (256/270)
Screening specificity (healthy images predicted as healthy)	0.878 (79/90)	0.889 (80/90)	0.911 (82/90)

Diseased images predicted as healthy (FN)	7	11	14
Healthy images predicted as diseased (FP)	11	10	8

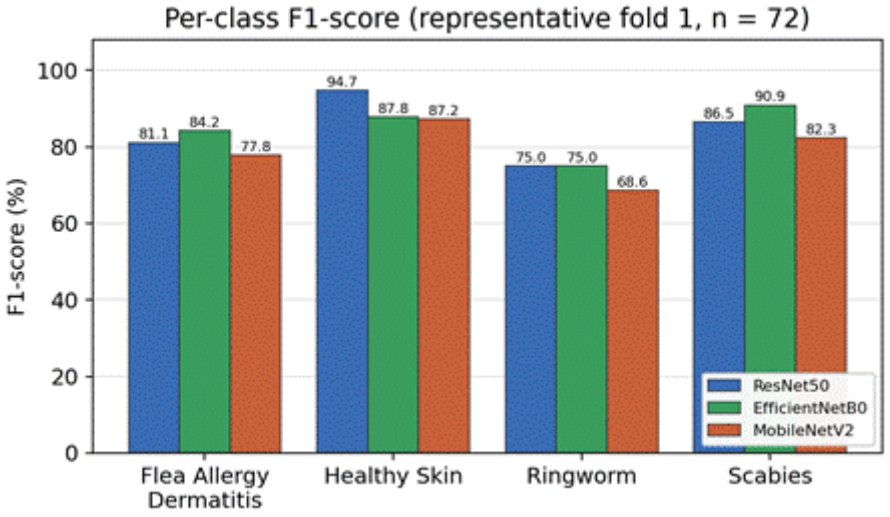


Figure 4.5. Per-class F1-score by architecture (pooled out-of-fold predictions, n = 360).

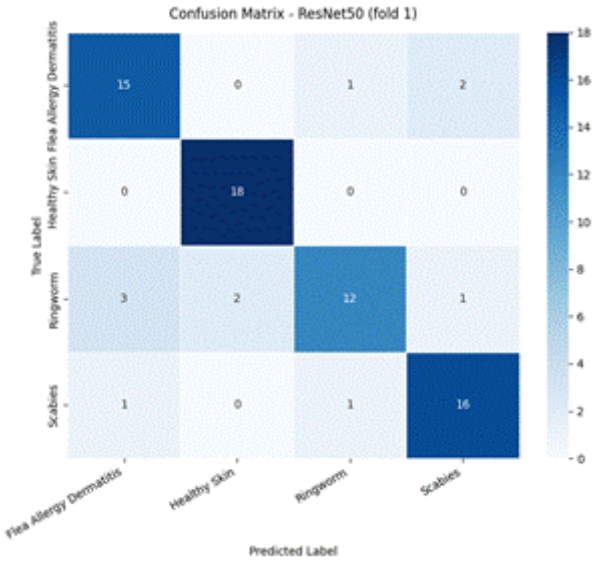


Figure 4.5a. Confusion matrix, ResNet50 (representative fold 1, n = 72).

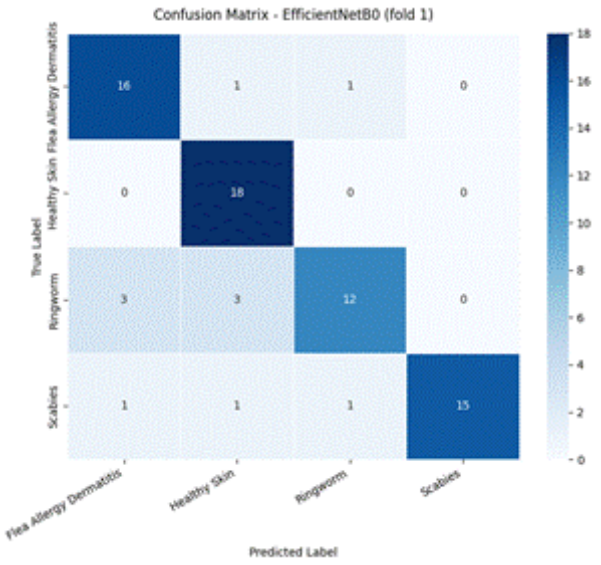


Figure 4.5b. Confusion matrix, EfficientNetB0 (representative fold 1, n = 72).

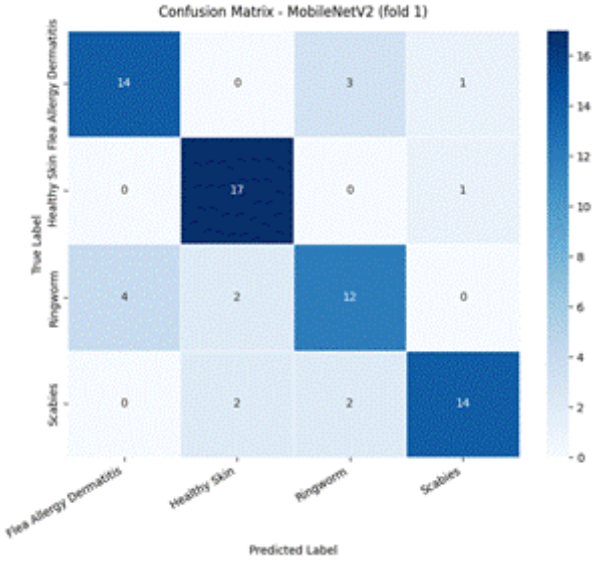


Figure 4.5c. Confusion matrix, MobileNetV2 (representative fold 1, n = 72).

In fold 1, ResNet50 and EfficientNetB0 each made 11 errors and MobileNetV2 made 15. Ringworm accounted for the largest share of errors in every model (6 of 11, 6 of 11, and 6 of 15). Misclassified ringworm images were predicted mostly as flea allergy dermatitis (3 for

ResNet50, 3 for EfficientNetB0, and 4 for MobileNetV2) and otherwise as healthy skin (2, 3, and 2), with one ringworm image predicted as scabies by ResNet50. Because all four classes have the same number of images, the difficulty with ringworm cannot be explained by class size. The more plausible causes are the overlap between ringworm's patchy hair loss with scaling and the hair loss and crusting of flea allergy dermatitis, and possible label noise; ringworm lesions that occupy only a small part of a photograph may also leave the visible coat looking normal, which would explain predictions of healthy skin. These explanations were not tested directly. The expert evaluation gives partial support to the label-noise explanation: the veterinarian rated label consistency lower for the Ringworm folder (4) than for the Scabies and Healthy Skin folders (5) (Section 3.2). The validator's low rating of image clarity (2 of 5) may also contribute, since small or unclear lesions are harder to recognize. Conversely, the validator named the healthy skin and scabies photographs as the most reliable part of the dataset, which agrees with scabies and healthy skin having the highest pooled F1-scores (Table 4.2). A similar pattern was reported by Aji et al. [8], whose MobileNetV2 model had the lowest recall for the fungal (ringworm) class (82%), with most errors predicted as flea infestation or scabies, which they attributed to similar skin texture.

Screening view. Collapsing the four classes into diseased and healthy (Table 4.2a) shows how often a diseased cat would be told that its skin looks healthy, the most costly error for a screening tool. Of the 270 diseased images, ResNet50 predicted 7 as healthy (screening sensitivity 0.974), EfficientNetB0 11 (0.959), and MobileNetV2 14 (0.948). Of the 90 healthy images, the order was reversed: ResNet50 predicted 11 as diseased (screening specificity 0.878), EfficientNetB0 10 (0.889), and MobileNetV2 8 (0.911). The architectures therefore differed in which screening error they made more often; these differences of a few images were not tested for significance. Over the pooled predictions, ringworm images made up half or more of the diseased images predicted as healthy: 4 of 7 for ResNet50, 6 of 11 for

EfficientNetB0, and 7 of 14 for MobileNetV2, whose other 7 were scabies images. Fold 1 showed the same pattern (2 of 2, 3 of 5, and 2 of 4).

Grad-CAM heatmaps [36] were generated for one test image of each class in fold 1 for each model (Figure 4.6). All twelve predictions shown were correct. For the ringworm example, all three models concentrated on the visible lesion, most sharply ResNet50 and EfficientNetB0. For the flea allergy dermatitis example, ResNet50 and EfficientNetB0 focused on patches of affected skin, while MobileNetV2's heatmap spread over most of the skin area; EfficientNetB0 also responded to the top-right corner near the gray letterbox padding. For the scabies example, ResNet50 and MobileNetV2 focused on the head, and EfficientNetB0 focused on the ears, the site where feline scabies typically appears [4], although its heatmap also spread into the background. For the healthy skin example, all three models focused on the cat's face and chest. That example is a full-body photograph of a cat on a sofa, whereas the disease examples are close-ups of affected skin, so the healthy prediction may rest partly on how the photograph is framed rather than on the skin itself. Because only four correctly classified images per model were inspected, and no misclassified images, this Grad-CAM analysis is illustrative only: it cannot show whether the models generally rely on lesions or on background, framing, source, or padding cues. A systematic Grad-CAM review, including the misclassified ringworm images, was not performed.

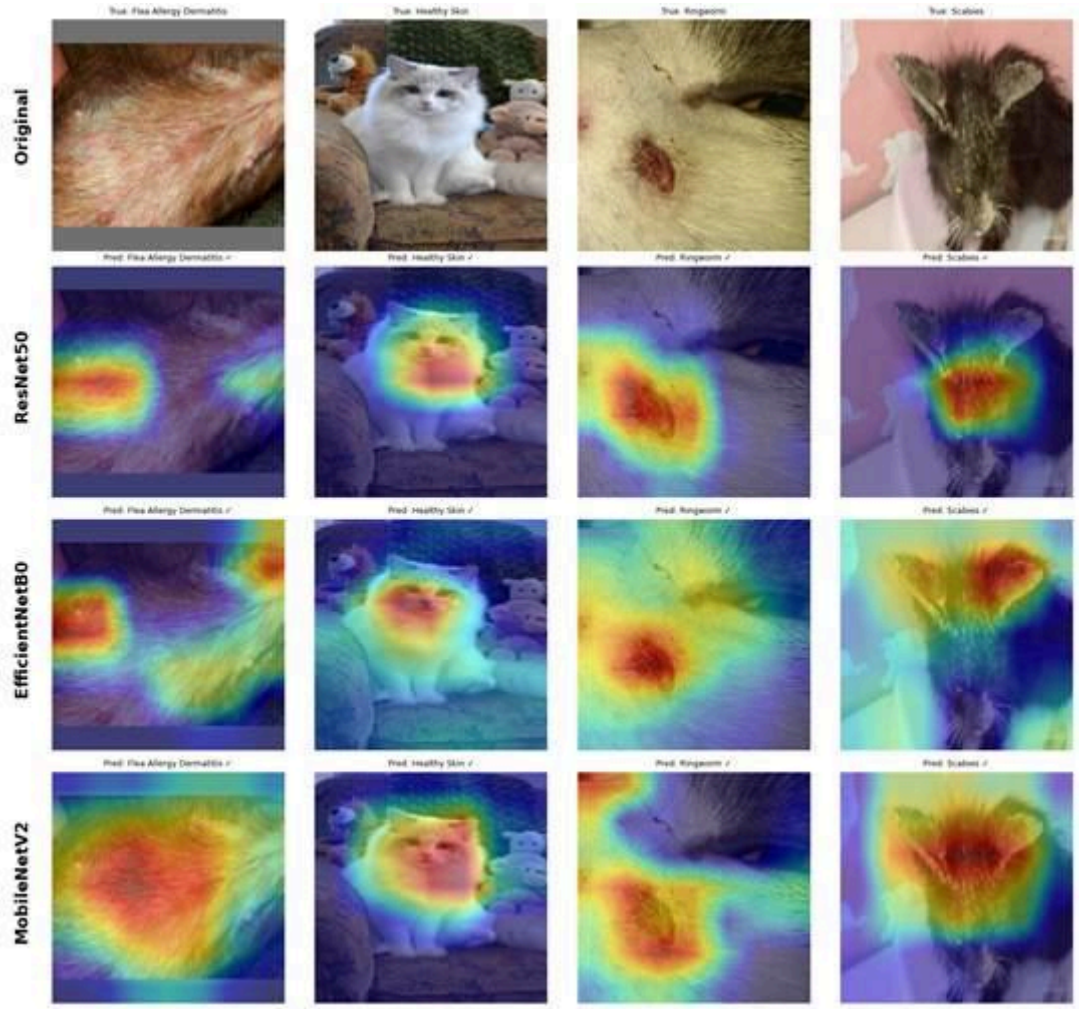


Figure 4.6. Grad-CAM heatmaps for one fold-1 test image of each class (top row: original letterboxed images; lower rows: ResNet50, EfficientNetB0, MobileNetV2).

4.3.3 Computational Characteristics

This section reports the computational characteristics of the three architectures (RQ1 and RO1). The architectures differ far more in computational characteristics than in classification accuracy (Table 4.3, Figure 4.7). MobileNetV2 is the lightest (2.59M parameters, 0.61 GFLOPs), followed by EfficientNetB0 (4.38M parameters, 0.80 GFLOPs); ResNet50 (24.11M parameters, 7.75 GFLOPs) has about 5.5 and 9.3 times as many parameters as EfficientNetB0 and MobileNetV2, respectively, and requires about 9.7 and 12.6 times as many FLOPs. In the final (Phase 2) models, 17,710,340 of ResNet50's

parameters were trainable, compared with 3,383,584 for EfficientNetB0 and 2,052,804 for MobileNetV2. Single-image inference time in eager execution did not fully follow FLOPs: MobileNetV2 was the fastest (191.3 ms), but EfficientNetB0 (346.7 ms) was slower than ResNet50 (270.3 ms) despite needing about one tenth of the operations. In graph mode, however, the order followed FLOPs: MobileNetV2 6.8 ms, EfficientNetB0 9.8 ms, and ResNet50 11.7 ms. The eager-mode order therefore mainly reflects per-layer framework overhead, which is largest for EfficientNetB0 (238 layers), rather than the computation itself. Even in graph mode, ResNet50 needed 12.6 times the FLOPs of MobileNetV2 but was only about 1.7 times slower, which is consistent with evidence that FLOPs are only an indirect measure of speed, which also depends on memory access cost and the hardware platform [35]; Aminah et al. [7] likewise measured EfficientNetB0 as slower than MobileNetV2 on a CPU (140 vs. 80 ms per image). Saved with optimizer state, the model files were 227.7, 43.3, and 26.1 MB; saved for inference only, they were 92.6, 17.5, and 10.4 MB, or about two-fifths of the size (calculated from the notebook's byte counts). Parameters and FLOPs remain the more reliable indicators of computational demand here.

All measured inference times (about 190 to 350 ms per image) are high for a GPU. They were measured by calling each model directly in eager execution, so they likely include substantial framework overhead and should be treated as relative, environment-specific figures. The per-image standard deviations were ±40.3 ms for ResNet50, ±51.8 ms for EfficientNetB0, and ±31.0 ms for MobileNetV2. These times come from the notebook's main timing, in which all three models were timed in a single session. A second timing with the same procedure in the same session gave 271.2 ± 41.5, 404.6 ± 258.6, and 169.3 ± 8.0 ms, so eager-mode times varied noticeably between repeated runs, especially for EfficientNetB0 and MobileNetV2. A supplementary graph-mode timing (tf.function, 20 warm-up and 3 × 200 timed passes per model) gave 11.7 ± 1.0 ms for ResNet50, 9.8 ± 2.0 ms for EfficientNetB0,

and 6.8 ± 1.4 ms for MobileNetV2. Parameter counts include the shared four-class classification head (328,964 parameters for MobileNetV2 and EfficientNetB0 and 525,572 for ResNet50; Section 3.6.3). Measured FLOPs are about twice the multiply-accumulate figures in the original architecture papers because the profiler counts multiplications and additions separately (Section 3.6.3). Parameters, FLOPs, and model size depend only on the architecture and head, not on the fold; inference time was measured on the fold-1 models.

Table 4.3. Computational characteristics of the three architectures.

Metric	ResNet50	EfficientNetB0	MobileNetV2
Total parameters	24,113,284	4,378,535	2,586,948
Trainable parameters, Phase 2 (final model)	17,710,340	3,383,584	2,052,804
Non-trainable parameters, Phase 2 (final model)	6,402,944	994,951	534,144
Trainable parameters, Phase 1 (frozen backbone)	525,572	328,964	328,964
FLOPs (GFLOPs)	7.75	0.80	0.61
Model size, incl. optimizer state (MB)*	227.7	43.3	26.1

Model size, inference only (MB)*	92.6	17.5	10.4
Inference (ms), mean ± SD†	270.3 ± 40.3	346.7 ± 51.8	191.3 ± 31.0

* Size of the saved .keras file in MB (bytes divided by 1,024²). "Incl. optimizer state" is the file saved as in the training procedure, which also stores the Adam optimizer state; "inference only" is the same model and weights saved without optimizer state (calculated from the byte counts printed by the notebook: 97,099,442, 18,344,985, and 10,924,229 bytes). Trainable and non-trainable counts are from Keras' weight lists; non-trainable weights include the frozen backbone layers and batch-normalization statistics. † Mean ± SD of 50 single-image forward passes after 5 warm-up passes, batch size 1, Google Colab Tesla T4 GPU, eager execution, all three models timed in the same session (the notebook's main timing). Graph-mode times are given in the text.

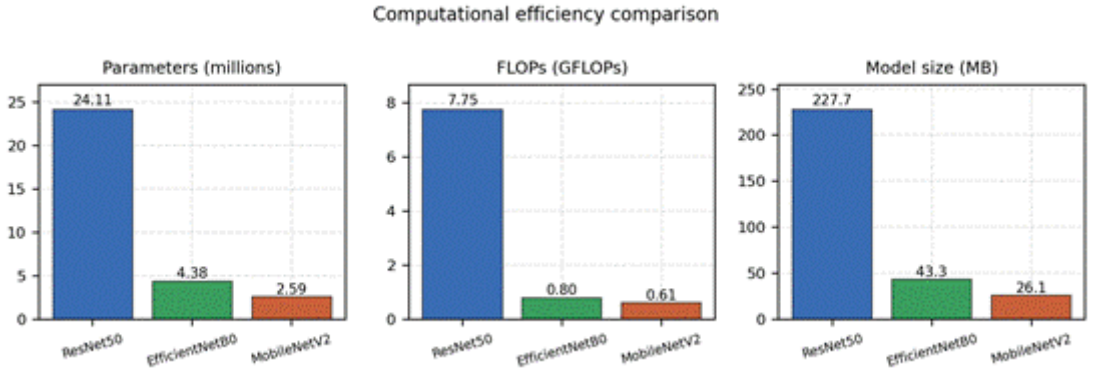


Figure 4.7. Parameters, FLOPs, and model size across the three architectures (model size includes optimizer state).

4.3.4 Summary of Findings for Objective 1

In answer to RQ1 and RO1, the three architectures reached moderate classification performance on the four classes: mean accuracy was 86.11% for ResNet50, 84.72% for EfficientNetB0, and 81.67% for MobileNetV2, with macro-F1 of 0.815 to 0.858 and MCC of 0.761 to 0.820 (Table 4.1). Ringworm was the hardest class for every architecture (F1 0.735 to 0.798), and screening sensitivity ranged from 0.948 to 0.974 and screening specificity from 0.878 to 0.911 (Tables 4.2 and 4.2a). Computationally, the architectures ranged from 2.59 million parameters and 0.61 GFLOPs (MobileNetV2) through 4.38 million and 0.80 GFLOPs (EfficientNetB0) to 24.11 million and 7.75 GFLOPs (ResNet50), with inference-only model files of 10.4, 17.5, and 92.6 MB (Table 4.3). Measured inference time followed FLOPs in graph mode but not in eager execution (Section 4.3.3).

4.4 Objective 2: Comparison of the Three Architectures

RO2 is to compare the three architectures on the results of RO1 and to test the pairwise differences in classification accuracy for statistical significance. Following the procedure in Section 3.7, Section 4.4.1 compares the observed results, Section 4.4.2 reports the statistical tests, and Section 4.4.3 summarizes the answer to RQ2 and RO2.

4.4.1 Descriptive Comparison

Classification performance. ResNet50 ranked first on every overall metric, EfficientNetB0 second, and MobileNetV2 third, in both the fold means and the pooled results (Table 4.1). Across folds, ResNet50 was the most accurate model in folds 2, 3, and 5, EfficientNetB0 in fold 4, and the two tied in fold 1 (84.72% each); MobileNetV2 was never the most accurate (Table 4.1a). Pooled over all 360 out-of-fold predictions (each image tested exactly once), ResNet50 classified 310 images correctly, EfficientNetB0 305, and MobileNetV2 294, so the observed differences in accuracy were 1.39 percentage points (5 images) between ResNet50 and EfficientNetB0, 4.44 points (16 images) between ResNet50 and MobileNetV2, and 3.06

points (11 images) between EfficientNetB0 and MobileNetV2 (calculated from the pooled counts). The 95% bootstrap confidence intervals of pooled accuracy overlap for all three pairs (Table 4.1), but overlapping intervals are not a formal test [33]; the formal test is reported in Section 4.4.2. At the class level, ResNet50 had the highest F1-score for ringworm, scabies, and healthy skin, EfficientNetB0 for flea allergy dermatitis, and MobileNetV2 the lowest in every class (Table 4.2). In screening, the architectures traded one error for another: ResNet50 predicted the fewest diseased images as healthy (7 of 270), and MobileNetV2 the fewest healthy images as diseased (8 of 90) (Table 4.2a).

Computational characteristics. The order was reversed for computational demand: MobileNetV2 was the lightest, followed by EfficientNetB0, while ResNet50 had about 5.5 and 9.3 times as many parameters and about 9.7 and 12.6 times as many FLOPs as EfficientNetB0 and MobileNetV2, respectively (Table 4.3). Inference time followed this order in graph mode (6.8, 9.8, and 11.7 ms), but in eager execution EfficientNetB0 was the slowest (Section 4.3.3). Unlike the accuracy differences, these differences are fixed properties of the architectures and do not vary between folds.

4.4.2 Statistical Comparison of Classification Accuracy

This section answers the inferential part of RQ2 and RO2, using the procedures in Section 3.6.2.

Exploratory fold-1 tests. McNemar's test was also applied to the 72 test images of fold 1, using the exact binomial test because every pair had fewer than 25 disagreements (Section 3.6.2). These results are exploratory and are not the RQ2 result. They showed no significant difference for any pair. For ResNet50 and EfficientNetB0, n01 = 4, n10 = 4, and p = 1.000. For ResNet50 and MobileNetV2, n01 = 7, n10 = 3, and p = 0.344. For EfficientNetB0 and MobileNetV2, n01 = 7, n10 = 3, and p = 0.344. The null hypothesis of equal error rates was

therefore not rejected for any pair at $\alpha = 0.05$, and because the smallest p-value was 0.344, a correction for the three comparisons would not change this. A test on 72 images has low statistical power.

Pooled comparison (RQ2 result). Table 4.4 reports the paired McNemar tests on all 360 pooled out-of-fold predictions. ResNet50 and MobileNetV2 disagreed on 52 images: 34 were classified correctly only by ResNet50 and 18 only by MobileNetV2 (raw $p = 0.038$). ResNet50 and EfficientNetB0 disagreed on 47 images (26 and 21; raw $p = 0.560$), and MobileNetV2 and EfficientNetB0 on 49 images (19 and 30; raw $p = 0.153$). After Holm correction for the three comparisons, the adjusted p-values were 0.113, 0.560, and 0.306, so the null hypothesis of equal error rates was not rejected for any pair at $\alpha = 0.05$. The ResNet50–MobileNetV2 difference was significant only before correction, so it is reported as not significant. For ResNet50 and EfficientNetB0, the difference of $d = 310 - 305 = 5$ correctly classified images also means that the exact test (Equation 3.14) could not give p below 0.0625, whatever the pattern of disagreement (Section 3.6.2), so this difference could not have been significant at $\alpha = 0.05$ on these data. A non-significant result does not show that two architectures are equivalent [34].

Table 4.4. Pairwise McNemar tests of classification accuracy on the pooled out-of-fold predictions (n = 360), with Holm correction.

Comparison (A vs B)	n01 (A correct, B wrong)	n10 (A wrong, B correct)	Test	Raw p‡	Holm-adjusted p‡	Significant at $\alpha = 0.05$
ResNet50 vs MobileNetV2	34	18	χ^2, continuity-co	0.038	0.113	No

			rrected			
ResNet50 vs EfficientNetB0	26	21	χ^2, continuity-corrected	0.560	0.560	No
MobileNetV2 vs EfficientNetB0	19	30	χ^2, continuity-corrected	0.153	0.306	No

‡ Continuity-corrected chi-square (Equation 3.15), the variant the notebook selected because n01 + n10 ≥ 25, with Holm step-down adjustment [54]; values as printed by the notebook and checked there against statsmodels.

4.4.3 Summary of Findings for Objective 2

In answer to RQ2 and RO2, the architectures differed in observed classification performance in the order ResNet50, EfficientNetB0, and MobileNetV2, and in computational demand in the reverse order, but no architecture was shown to be statistically superior in classification accuracy. None of the three pairwise differences was significant after Holm correction on the pooled predictions (Table 4.4), and none was significant on fold 1. The observed gaps of 5, 11, and 16 images out of 360 (Section 4.4.1) therefore do not establish that any architecture classifies more accurately than another, and whether the larger gaps involving MobileNetV2 matter in practice depends on the intended use and cannot be judged from significance tests alone. By contrast, the differences in computational characteristics are large, fixed properties of the architectures (Section 4.4.1).

4.5 Objective 3: Strengths and Weaknesses of Each Architecture

This section answers RQ3 and RO3 by considering classification performance and computational characteristics together, following the criteria in Section 3.7.3. Per-class and screening results are those pooled over all 360 out-of-fold predictions (Section 4.3.2).

ResNet50. Its classification strengths are the highest observed mean accuracy (86.11%), macro-averaged F1-score (0.858), and MCC (0.820). It was the most accurate model in three of the five folds and tied in a fourth. It also predicted the fewest diseased images as healthy (7 of 270; screening sensitivity 0.974) and had the highest ringworm recall (0.833) and F1-score (0.798). Its classification weaknesses are that its accuracy advantage over the other two architectures was not statistically significant (Section 4.4.2), and that ringworm remained its weakest class. It also predicted the most healthy images as diseased (11 of 90; screening specificity 0.878), and its fold accuracies ranged from 80.56% to 93.06%. Computationally, it was faster than EfficientNetB0 under eager execution (270.3 ms per image), but slower in graph mode (11.7 vs. 9.8 ms). However, it is by far the most demanding architecture: 24.11 million parameters, 7.75 GFLOPs, and a 92.6 MB inference-only model file (227.7 MB with optimizer state), or about 5.5 times the parameters and 9.7 times the FLOPs of EfficientNetB0. In practice, ResNet50 is suited to settings where computational resources are not constrained. Its higher observed accuracy did not translate into a statistically supported advantage over EfficientNetB0 on these data.

EfficientNetB0. Its classification strengths are an accuracy (84.72%) that is not significantly different from that of ResNet50, and the lowest observed fold-to-fold variability of the three models (±3.83 percentage points, only slightly below ResNet50's ±4.12 over five folds). It also had the highest pooled F1-score for flea allergy dermatitis (0.862) and the highest scabies precision (0.918). Its classification weaknesses are that it ranked second on every overall metric and on both screening measures (11 of 270 diseased images predicted as healthy, screening sensitivity 0.959; screening specificity 0.889), and that its ringworm

F1-score (0.754) was well below those of its other classes. Its computational strengths are about one fifth of the parameters of ResNet50 (4.38 million), about one tenth of its FLOPs (0.80 GFLOPs), and a 17.5 MB inference-only model file (43.3 MB with optimizer state). Its computational weakness is the slowest and most variable inference time under eager execution (346.7 ± 51.8 ms), despite its low FLOPs; in graph mode it was faster than ResNet50 (9.8 vs. 11.7 ms), so this weakness reflects framework overhead in the test environment more than the architecture's computation (Section 4.3.3). In practice, EfficientNetB0 offers classification accuracy statistically indistinguishable from that of ResNet50 at a small fraction of its parameters and FLOPs. Its inference time on a target device would need to be checked before it is used.

MobileNetV2. Its overall classification performance was the weakest observed. It had the lowest mean accuracy (81.67%), macro-averaged F1-score (0.815), and MCC (0.761) and the largest fold-to-fold variability (±5.08 percentage points; 75.00% to 90.28%). It was never the most accurate model in any fold, had the lowest pooled F1-score in every class (ringworm 0.735), and predicted the most diseased images as healthy (14 of 270; screening sensitivity 0.948). Its classification strengths are the highest healthy skin recall (0.911) and the highest screening specificity (0.911; 8 of 90 healthy images predicted as diseased). Its accuracy deficit was not statistically significant after Holm correction (Section 4.4.2). Its computational strengths are the smallest parameter count (2.59 million), the fewest FLOPs (0.61 GFLOPs), the smallest model file (26.1 MB with optimizer state; 10.4 MB for inference only), and the fastest and most consistent measured inference time (191.3 ± 31.0 ms in eager execution; 6.8 ms in graph mode). In practice, MobileNetV2 suits the most constrained settings, provided its lower observed accuracy is acceptable for the intended use.

Summary of findings for Objective 3. In answer to RQ3 and RO3, no architecture was strongest on every criterion. The architecture with the highest observed accuracy, ResNet50,

is the most computationally demanding. The lightest and fastest, MobileNetV2, had the lowest observed accuracy. EfficientNetB0 lies between them in observed accuracy and close to MobileNetV2 in parameters and FLOPs, but it had the slowest inference time under eager execution, though not in graph mode. Because no accuracy difference was significant, the clearest differences between the architectures lie in their computational characteristics. In screening, they traded one error for another: ResNet50 predicted the fewest diseased images as healthy, while MobileNetV2 predicted the fewest healthy images as diseased. The study does not compute a composite score or declare a single best architecture, because the relative weight of classification accuracy and computational demand depends on the intended use (Section 3.7.3).

4.6 Discussion

The discussion follows the three research objectives.

Objective 1 (classification performance and computational characteristics). The RO1 results (Section 4.3.4) show that all three architectures reached a moderate level of accuracy on the four classes, with the same class-level pattern, while their computational characteristics differed by close to an order of magnitude. This contrast between a small spread in accuracy and a large spread in computational demand is what Objectives 2 and 3 examine.

Two further findings bear on the weaknesses shared by all three architectures. First, ringworm remained the hardest class for every model even though all classes had the same number of images, so class size does not explain the difficulty; visual overlap with flea allergy dermatitis and possible label noise are the more plausible contributors, and the expert evaluation, which rated label consistency lower for the Ringworm folder and image clarity low, partly supports the second (Section 3.2). This points to targeted collection of clearer, veterinarian-verified ringworm images rather than a change of architecture. Second, the

healthy skin class largely worked as intended. Over the 360 pooled predictions, healthy skin recall was 0.878 to 0.911, and 7 to 14 of the 270 diseased images were predicted as healthy. The Grad-CAM example, however, suggests that healthy images may be recognized partly by their framing, so this result must be confirmed on healthy images photographed in the same way as the disease images (Section 5.2).

Objective 2 (comparison and statistical significance). The high capacity ResNet50 and the lightweight EfficientNetB0 reached accuracies whose difference was small and cannot be statistically significant (Sections 3.6.2 and 4.4.2): ResNet50's advantage was five images out of 360, so its extra capacity did not give a reliable accuracy advantage with this amount of data. A similar pattern has been reported elsewhere: on human skin images, ResNet-50 and EfficientNet-B0 performed about the same [21], and on a very small pet dataset a small custom CNN outperformed ResNet50 [18]. MobileNetV2's deficit relative to ResNet50 was significant only before Holm correction (Section 4.4.2), so it is reported as an observed difference, not a confirmed one.

Objective 3 (strengths and weaknesses). Because no accuracy difference was significant while the computational differences were large, the trade-off between the architectures lies mainly in computational demand. For RQ3, this means that the larger architecture's extra computational demand was not matched by a statistically supported gain in classification accuracy (Section 4.5). The choice between the architectures therefore depends on the intended use (Section 3.7.3): where computational resources are constrained, EfficientNetB0 and MobileNetV2 need a small fraction of ResNet50's parameters and FLOPs, and where missing a diseased cat is the main concern, ResNet50's lower number of diseased images predicted as healthy is an observed difference that was not tested for significance (Section 4.3.2).

Comparison with prior studies. Direct numerical comparison with the prior feline studies is limited. Aji et al. [8] reported 91.8% validation accuracy for MobileNetV2 on 1,600 images in four classes (including healthy skin), using a single 80/20 train-validation split with no separate test set. Aminah et al. [7] reported 99.38 to 99.79% test accuracy for MobileNetV2, EfficientNetB0, and EfficientNetV2B0 on nine classes; however, their images were augmented offline and the augmented pool was then divided into training, validation, and test sets, and the paper does not state that rotated and flipped copies of the same photograph were kept in the same subset, so part of that accuracy may reflect the kind of near-duplicate overlap shown to inflate results [10]. Syahputra and Arifudin reported 92% accuracy for MobileNetV2 on six classes, also from a single fixed split [9]. The present accuracies (81.67 to 86.11%) were obtained on four classes after a near-duplicate check and with every image tested exactly once in cross-validation, so they are not directly comparable. Consistent with Newaz et al. [21], who found ResNet-50 and EfficientNet-B0 comparable and MobileNetV2 weaker on a large mobile-image dataset, the present results show no significant accuracy advantage for the largest architecture over EfficientNetB0, with MobileNetV2 lowest in mean accuracy.

4.7 Interpretation of the Final Accuracy

The final accuracies (81.67 to 86.11%) should be read in the context of the study's data and design rather than as a ceiling on what the architectures can achieve. Three factors are the most likely contributors to the level reached. First, the dataset is small, 360 images with held-out test folds of 72 images, which both limits the features a model can learn and widens the confidence intervals around every estimate (Section 4.3.1). Second, the labels were drawn from public repositories. The veterinarian who evaluated the dataset judged it valid with corrections, rated label consistency for the Ringworm folder below that of the Scabies and Healthy Skin folders, and rated the clarity of the photographs below neutral

(Section 3.2). Part of the residual error may therefore reflect label noise and image quality rather than model error, particularly for the visually confusable ringworm and flea allergy dermatitis classes. Third, the disease classes overlap visually by nature, so some misclassification is expected even with clean labels; this is visible in the concentration of errors in the ringworm class (Section 4.3.2). Accuracy at this level is consistent with the available data.

The training record supports this reading. All three models reached close to 100% training accuracy while inner-validation and test accuracy stayed lower (Section 4.2), which shows that the models could fit the training images but did not generalize equally well to new ones. In fold 1, the only fold whose training histories are shown, early stopping ended every Phase 2 run once inner-validation loss stopped improving, so more epochs under the same settings would not be expected to help. Fold-to-fold accuracy varied widely (Section 4.3.1; Table 4.1a), which shows how strongly the estimate depends on which 72 images are in the test fold. Same-cat leakage and any framing difference between healthy and diseased photographs, if present, would tend to inflate rather than lower these accuracies [11].

Chapter 5

CONCLUSION AND RECOMMENDATION

5.1 Conclusion

This study compared ResNet50, MobileNetV2, and EfficientNetB0 for classifying photographs of cat skin into four classes, flea allergy dermatitis, ringworm, scabies, and healthy skin, using the same 360 images, the same training procedure, and the same five-fold cross-validation for all three. The conclusions below answer the three research questions.

RQ1 and RO1 (classification performance and computational characteristics). ResNet50 had the highest mean accuracy (86.11% ± 4.12), followed by EfficientNetB0 (84.72% ± 3.83) and MobileNetV2 (81.67% ± 5.08), with the same order for macro-F1 (0.858, 0.846, and 0.815) and MCC (0.820, 0.801, and 0.761). Over the 360 pooled predictions, ringworm was the hardest class for every model (F1 0.735 to 0.798), which agrees with the visual overlap described in Section 2.1, while scabies and healthy skin had the highest F1-scores. In the disease-versus-healthy view, the models predicted between 7 (ResNet50) and 14 (MobileNetV2) of the 270 diseased images as healthy, and between 8 (MobileNetV2) and 11 (ResNet50) of the 90 healthy images as diseased. In computational characteristics, ResNet50 had 24.11 million parameters and 7.75 GFLOPs, compared with 4.38 million and 0.80 GFLOPs for EfficientNetB0 and 2.59 million and 0.61 GFLOPs for MobileNetV2. The saved model files were 227.7, 43.3, and 26.1 MB including optimizer state, and 92.6, 17.5, and 10.4 MB for inference only. Measured inference time under eager execution did not fully follow this order: MobileNetV2 was the fastest (191.3 ms per image) and EfficientNetB0 the slowest (346.7 ms); in graph mode the order followed FLOPs (6.8, 9.8, and 11.7 ms for MobileNetV2, EfficientNetB0, and ResNet50).

RQ2 and RO2 (differences and statistical significance). Descriptively, ResNet50 had the highest observed accuracy, macro-averaged F1-score, and MCC, EfficientNetB0 the second highest, and MobileNetV2 the lowest. For parameters, FLOPs, and model file size the order was reversed, with MobileNetV2 the lightest. Inferentially, paired McNemar tests on the 360 pooled predictions found no significant difference for any pair after Holm correction. The adjusted p-values were 0.113 for ResNet50 and MobileNetV2, 0.306 for MobileNetV2 and EfficientNetB0, and 0.560 for ResNet50 and EfficientNetB0; the ResNet50–MobileNetV2 difference was significant only before correction (raw $p = 0.038$). No architecture was therefore shown to be statistically superior in classification accuracy, although this does not show that the three are equivalent.

RQ3 and RO3 (strengths and weaknesses). ResNet50's strengths were the highest observed accuracy, macro-averaged F1-score, and MCC, and the fewest diseased images predicted as healthy. Its weaknesses were by far the largest parameter count, FLOPs, and model file size, without a significant accuracy advantage over either other architecture, and the most healthy images predicted as diseased. EfficientNetB0 combined accuracy not significantly different from that of ResNet50, and slightly lower fold-to-fold variability, with about one fifth of ResNet50's parameters and one tenth of its FLOPs. Its weaknesses were the slowest and most variable inference time under eager execution (not in graph mode), and second-ranked overall accuracy, macro-averaged F1-score, MCC, and screening measures. MobileNetV2 had the smallest size, the fewest FLOPs, the fastest measured inference, and the highest screening specificity. Its weaknesses were the lowest observed accuracy, the lowest F1-score in every class, the most diseased images predicted as healthy, and the largest variability. No architecture was strongest on every criterion; which is most appropriate depends on whether classification accuracy or computational demand matters more for a given use.

Overall, on this dataset the high capacity model did not give a significant accuracy advantage over the smaller EfficientNetB0. Adding a healthy skin class removed the earlier limitation that every image was assigned a disease, and the models separated healthy from diseased skin with screening sensitivity of 0.948 to 0.974 and screening specificity of 0.878 to 0.911, although part of this may reflect differences in photograph framing. The accuracy level reached (81.67% to 86.11%) is moderate and reflects a small dataset, labels from public sources, photographs of limited clarity, and visually similar disease classes. A veterinarian who evaluated the dataset judged it valid with corrections, agreeing that most photographs are correctly labeled and recommending clearer images; a concern about one image in the Ringworm folder was resolved in an in-person discussion with the validator. The results describe the dataset as it was evaluated. The models are not diagnostic tools.

5.2 Recommendations

The following recommendations come directly from the findings and limitations of this study.

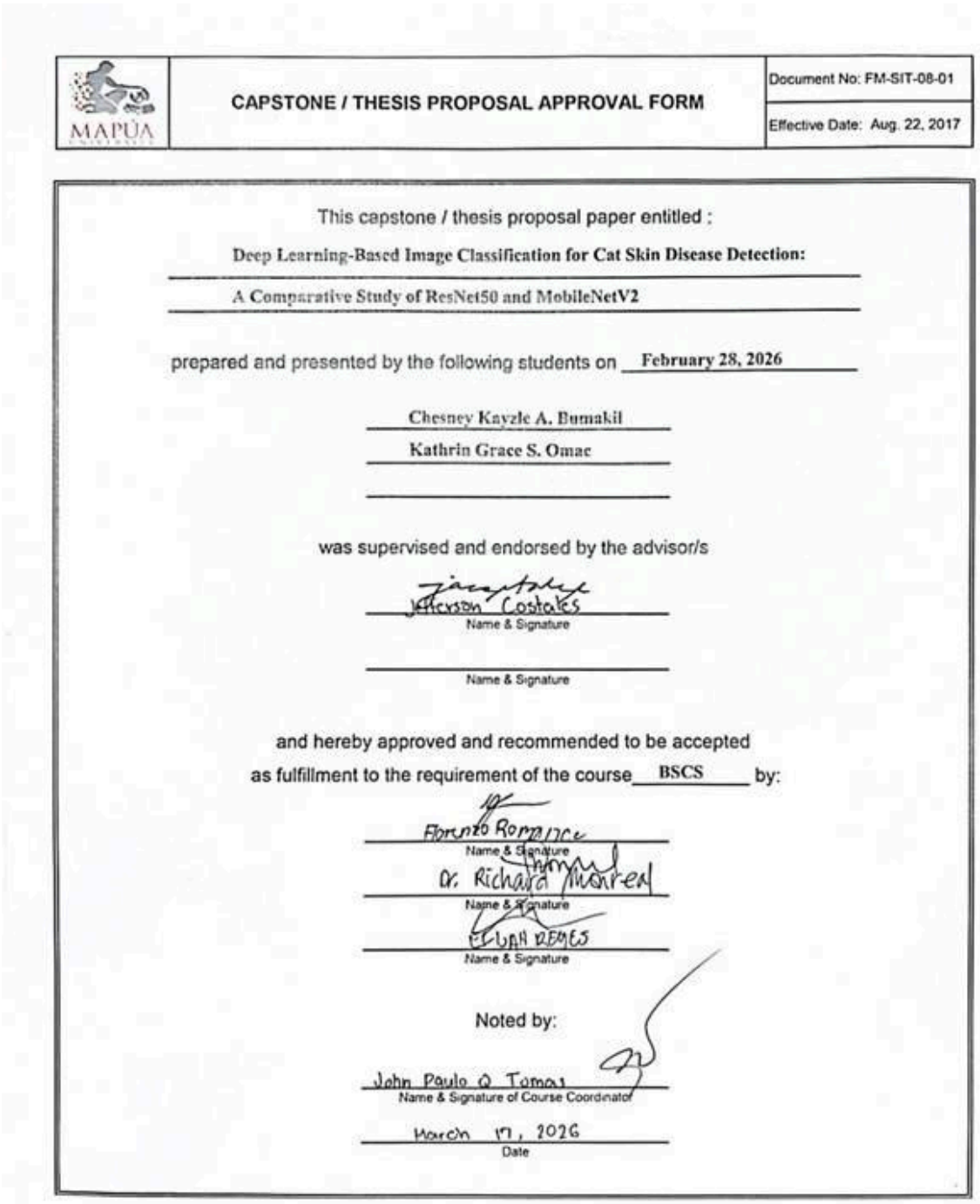
1. Expand and improve the dataset. Collect more and clearer images, as the expert validator recommended, especially of ringworm, with labels verified by a veterinarian and a record of which images show the same cat, so that data can be split by cat [11]. Record the breed, coat type, sex, and age of each cat, which neither the sources used nor the feline studies reviewed reported (Section 2.1.4), and label lesion severity, as Aminah et al. did with two severity levels [7], so that severity grading, which is outside the present scope (Section 1.6), can be studied. Collect healthy skin images framed in the same way as the disease images (close-ups of the same body regions), so that the healthy class cannot be recognized from framing alone. Adding an "other or unknown" class, or a rule that rejects low-confidence predictions [51], would also

keep the model from forcing an image of an unstudied condition into one of the four classes.

2. Extend the evaluation. Repeat the cross-validation with several random seeds, so that the comparison of classification accuracy also reflects variation between training runs, which the pooled McNemar tests do not capture [32]. Carry out a systematic Grad-CAM review of misclassified images, and test the three architectures on images from a different source, such as a veterinary clinic.
3. Refine the computational evaluation. Tune the hyperparameters of each architecture using nested cross-validation [26], so that each is compared at its best setting. Measure inference time on the intended device and outside eager execution, since FLOPs do not predict speed reliably [35] and the present timings include framework overhead (Section 4.3.3). Report the file size of each model in the format used on the target device. The comparison could also be extended to architectures outside the present scope, such as EfficientNetV2B0 and DenseNet (Section 2.3).

APPENDICES

Appendix A. Thesis Proposal Defense Approval Form



MAPÚA

CAPSTONE / THESIS PROPOSAL APPROVAL FORM

Document No: FM-SIT-08-01

Effective Date: Aug. 22, 2017

This capstone / thesis proposal paper entitled :

Deep Learning-Based Image Classification for Cat Skin Disease Detection:

A Comparative Study of ResNet50 and MobileNetV2

prepared and presented by the following students on **February 28, 2026**

Chesney Kayzle A. Bumakil

Kathrin Grace S. Omac

was supervised and endorsed by the advisor/s

Jefferson Costales
Name & Signature

Name & Signature

and hereby approved and recommended to be accepted

as fulfillment to the requirement of the course **BSCS** by:

Florenzo Romanca
Name & Signature

Dr. Richard Monreal
Name & Signature

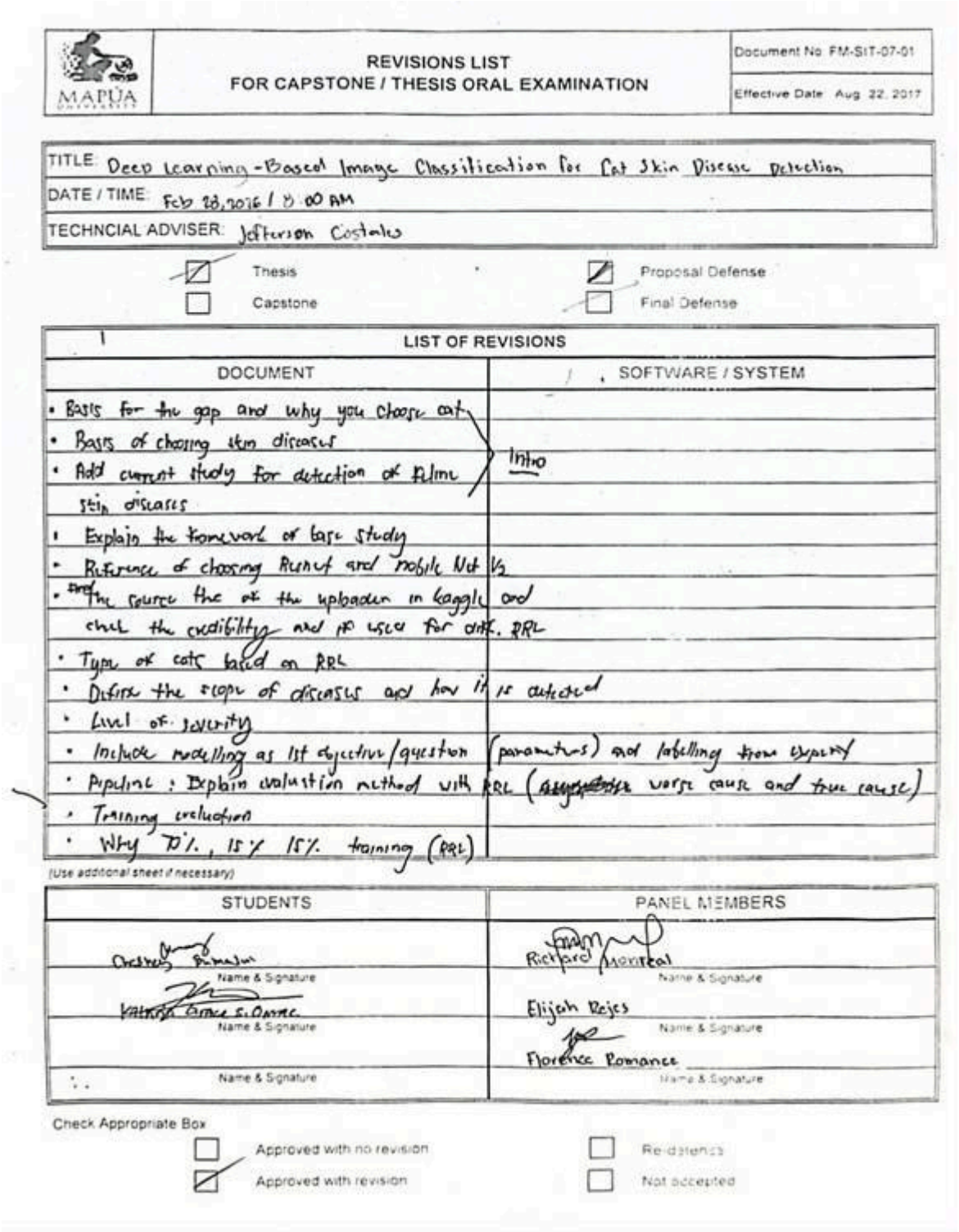
ELIJAH REYES
Name & Signature

Noted by:

John Paulo Q Tomas
Name & Signature of Course Coordinator

March 17, 2026
Date

Appendix B. Thesis Proposal Defense Revision List



MAPÚA UNIVERSITY

REVISIONS LIST
FOR CAPSTONE / THESIS ORAL EXAMINATION

Document No. FM-SIT-07-01
Effective Date: Aug. 22, 2017

TITLE: Deep Learning-Based Image Classification for Cat Skin Disease Detection
DATE / TIME: Feb 28, 2016 / 8:00 AM
TECHNCIAL ADVISER: Jefferson Costales

☒ Thesis ☐ Capstone ☒ Proposal Defense ☐ Final Defense

LIST OF REVISIONS

DOCUMENT	SOFTWARE / SYSTEM
• Basis for the gap and why you choose cat.	
• Basis of choosing skin diseases	
• Add current study for detection of feline skin diseases	Intro
• Explain the framework of base study	
• Reference of choosing Resnet and mobile Net V2	
• and the source the of the uploader in kaggle and check the credibility and ID used for diff. RRL	
• Type of cats based on RRL	
• Define the scope of diseases and how it is detected	
• Level of severity	
• Include modelling as 1st objective/question	(parameters) and labelling from experts
• Pipeline : Explain evaluation method with RRL (~~Augmentation~~ worse case and true cause)	
• Training evaluation	
• Why 70%, 15% 15% training (RRL)	

(Use additional sheet if necessary)

STUDENTS	PANEL MEMBERS
Name & Signature	Richard Montreal Name & Signature
Name & Signature	Elijah Rejes Name & Signature
Name & Signature	Florence Romance Name & Signature

Check Appropriate Box

☐ Approved with no revision ☐ Re-defense
☒ Approved with revision ☐ Not accepted

Appendix C. Thesis Proposal Defense Conformity of Revision

Date : 03/17/2026

TITLE **:** Deep Learning-Based Image Classification of Cat Skin Diseases: A Comparative Study of ResNet50, MobileNetV2, and EfficientNetB0

Name of Student	Student No.	Program of Study
Kathrin Grace S. Omac	2023108505	Computer Science
Chesney Kayzle A. Bumakil	2023100445	Computer Science

IMPLEMENTATION OF THE RECOMMENDED REVISIONS

Nature of Revision	Actions Made	Page Reflected Manuscript
Basis for the gap and why you chose cat.	Stated the research gap and why cats were chosen.	Sec. 1.1, 1.2, 2.5
Basis of choosing skin disease	Explained the basis for the three disease classes.	Sec. 2.1

Add current study for detection of feline Skin diseases	Added recent feline skin disease studies (Table 2.1).	Sec. 2.2; Table 2.1
Explain the framework of base Study	Explained the base studies' approach and the study framework (Figure 3.1).	Sec. 2.2, 3.1; Figure 3.1
Reference of choosing Resnet and MobileNetv2	Added references for choosing ResNet50, MobileNetV2, and EfficientNetB0.	Sec. 2.3
Find the source the at the uploader in kaggle and check the credibility and if used for diff RRL	Cited the Kaggle and Roboflow sources. Credibility check	Sec. 3.2; Appendices F, G
Type of cats based on RRL	Stated that breed and coat type are not recorded (limitation).	Sec. 1.6, 2.1.4, 5.2
Define the scope of diseases and how it is detected	Defined the four classes and how the model detects them.	Sec. 1.6, 3.5; Eq. 3.2
Should be able to tell the level of severity	Not included; images have no severity labels (stated in scope).	Sec. 1.6, 3.2, 5.2
Include modeling as 1st objective/question (parameters) and labelling from expert	Revised objectives to cover model performance and parameters; expert labelling stated as a limitation.	Sec. 1.3, 1.4, 1.6, 3.2; Appendices F, G

Pipeline: Explain evaluation method with RRL (worse cause and true cause)	Explained the evaluation method with supporting literature.	Sec. 2.4, 3.6, 3.6.4; Eqs. 3.5-3.26; Table 3.4
Training Evaluation	Added training curves and their analysis.	Sec. 4.2; Figs. 4.1-4.3
Why 70%. , 15%, 15% training (RRL)	Replaced the 70/15/15 split with stratified five-fold cross-validation, with literature basis.	Sec. 3.4; Table 3.2

THESIS/CAPSTONE CONFORMITY OF REVISIONS

Chesney Kayzle A. Bumakil
Student

Kathrin Grace S. Omac
Student

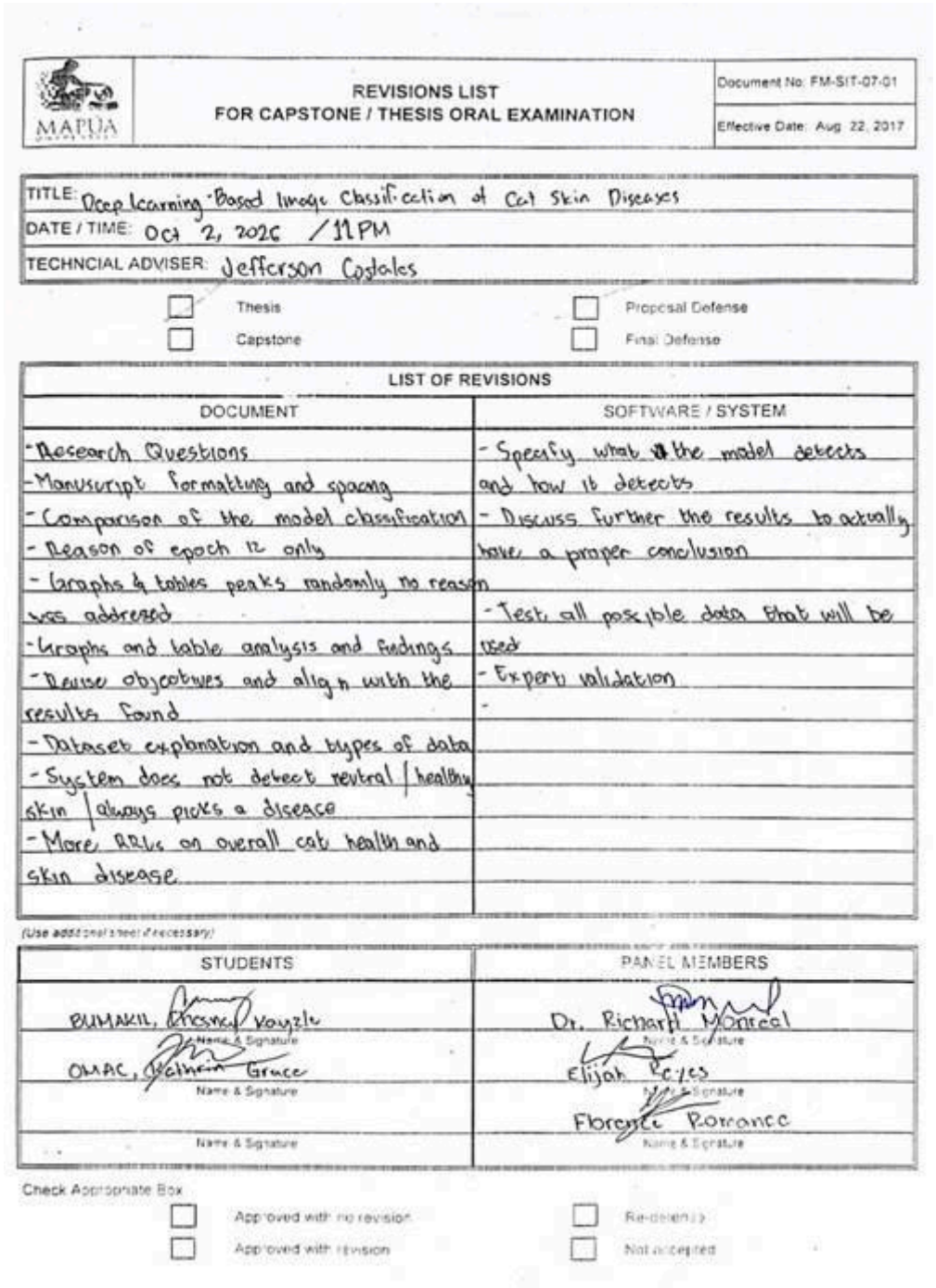
Jefferson Costales
Adviser

Richard Monreal
Panel Member

Florence Romance
Panel Member

Elijah Reyes
Panel Member

Appendix D. Thesis Final Defense Revision List



MAPUA

REVISIONS LIST
FOR CAPSTONE / THESIS ORAL EXAMINATION

Document No: FM-SIT-07-01
Effective Date: Aug. 22, 2017

TITLE: Deep learning-Based Image Classification of Cat Skin Diseases
DATE / TIME: Oct 2, 2025 / 11 PM
TECHNCIAL ADVISER: Jefferson Costales

☐ Thesis ☐ Proposal Defense
☐ Capstone ☐ Final Defense

LIST OF REVISIONS

DOCUMENT	SOFTWARE / SYSTEM
-Research Questions	- Specify what & the model detects
-Manuscript formatting and spacing	and how it detects
- Comparison of the model classification	- Discuss further the results to actually
- Reason of epoch 12 only	have a proper conclusion
- Graphs & tables peaks randomly no reason	
was addressed	- Test all possible data that will be
-Graphs and table analysis and findings	used
- Revise objectives and align with the	- Expert validation
results found	-
- Dataset explanation and types of data	
- System does not detect revtral / healthy	
skin / always picks a disease	
- More RRLs on overall cat health and	
skin disease	

(Use additional sheet if necessary)

STUDENTS	PANEL MEMBERS
BUMAKIL, Chesney Vouzle	Dr. Richard Monreal
Name & Signature	Name & Signature
OMAC, Nathan Grace	Elijah Reyes
Name & Signature	Name & Signature
	Florence Porrance
Name & Signature	Name & Signature

Check Appropriate Box

☐ Approved with no revision ☐ Re-defend
☐ Approved with revision ☐ Not accepted

Appendix E. Thesis Final Defense Conformity of Revision

Date : 10/2/2026

TITLE **:** Deep Learning-Based Image Classification of Cat Skin Diseases: A Comparative Study of ResNet50, MobileNetV2, and EfficientNetB0

Name of Student	Student No.	Program of Study
Kathrin Grace S. Omac	2023108505	Computer Science
Chesney Kayzle A. Bumakil	2023100445	Computer Science

IMPLEMENTATION OF THE RECOMMENDED REVISIONS

Nature of Revision	Actions Made	Page Reflected Manuscript
Research Questions	Added research questions RQ1 to RQ3.	Sec. 1.3, 5.1
Manuscript formatting and spacing	Fixed headings, captions, appendix labels, and body text fonts; updated all cross-references.	Whole manuscript

Comparison of model classification	Added per-fold, pooled, and statistical comparison of the models. Described the comparison procedure, evaluation metrics, and criteria in the Methodology, and reported the comparison results by objective in the Results and Discussion.	Sec. 3.6.2, 3.7, 4.3-4.5; Tables 4.1, 4.1a, 4.4
Reason for epoch stop	Explained early stopping, the earliest and latest possible stopping epochs, and the epoch at which each model stopped.	Sec. 3.5, 3.6.4, 4.2; Table 3.4
Graphs and tables peaks randomly with no reason addressed	Explained the peaks in the validation curves.	Sec. 4.2; Figs. 4.1-4.3
Graphs and tables analysis and findings	Added analysis after every table and figure.	Sec. 4.2 to 4.5
Revise objectives and align with the results found	Revised objectives and aligned conclusions with the results; organized the Results and Discussion by objective.	Sec. 1.3, 1.4, 4.3-4.6, 5.1
Dataset explanation and type of data	Expanded the dataset description, type of data, sources, and the credibility of the sources.	Sec. 3.2; Table 3.1
System does not detect neutral/ healthy skin/ always picks a disease	Added a Healthy Skin class and re-ran the model.	Sec. 2.1.4, 3.2, 4.3.2; Tables 4.2, 4.2a

More RRLs on overall cat health and skin disease	Added literature on cat health and skin disease.	Sec. 1.1, 2.1.4
Specify what model detects and how it detects	Stated what the model detects (four classes, one label per photograph) and how (softmax output; Grad-CAM shows the image regions used).	Sec. 1.6, 3.5, 3.6.5; Eq. 3.2; Fig. 4.6
Discuss further the results to actually have a proper conclusion	Expanded the discussion and conclusions.	Sec. 4.4.2 to 5.2
Test all possible data that will be used	Tested all 360 images through five-fold cross-validation.	Sec. 3.4, 4.3.1; Table 4.1a
Expert Validation	A veterinarian evaluated the dataset and its labels; the ratings, the resolution of the concern about one Ringworm image, and the effect on the results are reported.	Sec. 3.2, 4.3.2, 4.7, 5.1; Appendices F, G

Appendix F. Expert Dataset Evaluation Form

Dataset Evaluation Form: Domain Expert

Deep Learning-Based Image Classification of Cat Skin Diseases: A Comparative Study of ResNet50, MobileNetV2, and EfficientNetB0

Researchers: Bumakil, Omac Adviser: Jefferson Costales Mapua Institute of Technology

What you are evaluating. A dataset of 360 photographs of cat skin collected from publicly available online sources and divided equally into four categories: flea allergy dermatitis, ringworm, scabies, and healthy skin (90 photographs each). Each photograph has one label, given by the folder it is stored in. The labels were reviewed during data preparation but not by a veterinarian. The dataset is used to train and test image classification models; it is not a clinical system.

Please complete this form after reviewing the photographs in the shared folders.

Rate each statement

Statement (1 = strongly disagree, 3 = neutral, 5 = strongly agree)	1	2	3	4	5	N/A
A. Labels per category						
1. Photos in the Flea Allergy Dermatitis folder show signs consistent with flea allergy dermatitis.	☐	☐	☐	☒	☐	☐
2. Photos in the Ringworm folder show lesions consistent with ringworm (dermatophytosis).	☐	☐	☐	☒	☐	☐
3. Photos in the Scabies folder show signs consistent with feline scabies (notoedric mange).	☐	☐	☐	☐	☒	☐
4. Photos in the Healthy Skin folder show cats with no visible signs of skin disease.	☐	☐	☐	☐	☒	☐
B. Image quality						
5. The photos are clear enough to judge the condition of the skin.	☐	☒	☐	☐	☐	☐
6. The affected skin area is visible in most photos.	☐	☐	☐	☒	☐	☐
C. Overall						
7. The four categories can generally be told apart from a photograph.	☐	☐	☐	☒	☐	☐
8. Most photos in the dataset are correctly labeled.	☐	☐	☐	☒	☐	☐
9. The dataset is suitable for a research study on classifying cat skin conditions from photographs.	☐	☐	☐	☒	☐	☐

Overall judgement of the dataset (tick one):

☐ Valid as labeled ☒ Valid with corrections ☐ Not valid in its present form

Most reliable part of the dataset:

Pictures of those w/ healthy skin & those w/ scabies.

Most important labeling error or concern observed (name the category and file names if possible):

Dermatophytosis : Atopic derma in dog (file name)

What would need to change for the dataset to be more reliable?

clearer images

Reviewer

Full name, title	LAIZA BANING, DVM	Institution	PAW HAUS VET CLINIC

Appendix G. Certificate of Validation

CERTIFICATE OF VALIDATION

This is to certify that the dataset used in the thesis entitled "Deep Learning-Based Image Classification of Cat Skin Diseases: A Comparative Study of ResNet50, MobileNetV2, and EfficientNetB0," conducted by Chesney Kayzle A. Bumakil and Kathrin Grace S. Omac, Bachelor of Science in Computer Science students of the School of Information Technology, Mapua Institute of Technology, has been evaluated and validated by the undersigned.

The dataset consists of 360 photographs of cat skin in four categories: flea allergy dermatitis, ringworm, scabies, and healthy skin (90 photographs each). The findings of the evaluation are recorded in the accompanying Dataset Evaluation Form.

Issued this 7th day of October 2026.

LAIZA CAMILLE H. BANING
Signature over Printed Name

Position/Title: RESIDENT VETERINARIAN

Institution: PAW HAUS VET CLINIC

License No. (if applicable): _____

REFERENCES

[1] Hill, P. B. et al. 2006. Survey of the prevalence, diagnosis and treatment of dermatological conditions in small animals in general practice. Veterinary Record. 158, 16 (2006), 533-539.

[2] Hobi, S., Linek, M., Marignac, G., Olivry, T., Beco, L., Nett, C., Fontaine, J., Roosje, P., Bergvall, K., Belova, S., Koebrich, S., Pin, D., Kovalik, M., Meury, S., Wilhelm, S., and Favrot, C. 2011. Clinical characteristics and causes of pruritus in cats: a multicentre study on feline hypersensitivity-associated dermatoses. Veterinary Dermatology. 22, 5 (2011), 406-413. https://doi.org/10.1111/j.1365-3164.2011.00962.x.

[3] Hwang, S., Shin, H. K., Park, J. M., Kwon, B., and Kang, M. G. 2022. Classification of dog skin diseases using deep learning with images captured from multispectral imaging device. Molecular & Cellular Toxicology. 18, 3 (2022), 299-309. https://doi.org/10.1007/s13273-022-00249-7.

[4] Sampaio, K. O., de Oliveira, L. M. B., Burmann, P. M., Sousa Filho, R. P., Evangelista, J. S. A. M., and Cunha, M. G. M. C. M. 2017. Acetate tape impression test for diagnosis of notoedric mange in cats. Journal of Feline Medicine and Surgery. 19, 6 (2017), 702-705. https://doi.org/10.1177/1098612X16632279.

[5] Pasteur, K., Diana, A., Yatcilla, J. K., Barnard, S., and Croney, C. C. 2024. Access to veterinary care: evaluating working definitions, barriers, and implications for animal welfare. Frontiers in Veterinary Science. 11 (2024), 1335410. https://doi.org/10.3389/fvets.2024.1335410.

[6] de Castro, J. L. et al. 2026. Digital engagement and knowledge about zoonoses among dog and cat owners in Rio de Janeiro: a cross-sectional study. Frontiers in Digital Health. 8 (2026), 1746563. https://doi.org/10.3389/fdgth.2026.1746563.

[7] Aminah, F. R. N., Mutasodirin, M. A., and Hidayattullah, M. F. 2025. Cat skin disease diagnosis using EfficientNetV2 for lightweight processing on low-resource devices. Jurnal Teknik Elektro. 17, 2 (2025), 45-50. https://doi.org/10.15294/jte.v17i2.29764.

[8] Aji, D. S., Ashari, W. M., and Ariyus, D. 2025. Classification of cat skin diseases using MobileNetV2 architecture with transfer learning. Journal of Applied Informatics and Computing. 9, 6 (2025), 3212-3219. https://doi.org/10.30871/jaic.v9i6.11469.

[9] Syahputra, S. N. and Arifudin, R. 2026. Application of fine-tuning on convolutional neural networks to improve classification accuracy of feline skin diseases. Jurnal Penelitian Pendidikan. 43, 1 (2026), 65-72. https://journal.unnes.ac.id/journals/JPP.

[10] Barz, B. and Denzler, J. 2020. Do we train on test data? Purging CIFAR of near-duplicates. Journal of Imaging. 6, 6 (2020), 41. https://doi.org/10.3390/jimaging6060041.

[11] Saeb, S., Lonini, L., Jayaraman, A., Mohr, D. C., and Kording, K. P. 2017. The need to approximate the use-case in clinical machine learning. GigaScience. 6, 5 (2017), 1-9. https://doi.org/10.1093/gigascience/gix019.

[12] Beohar, K., Battawar, D., and Vahreza, A. Cat Skin Disease v3 Dataset. Kaggle. Accessed February 2026. https://www.kaggle.com/datasets/vekified/cat-skin-disease-v3.

[13] Tain. 2022. Cat Ringworm Computer Vision Dataset (Version V3) [Dataset]. Roboflow Universe. https://universe.roboflow.com/tain/cat-ringworm.

[14] Santoro, D., Pucheu-Haston, C. M., Prost, C., Mueller, R. S., and Jackson, H. 2021. Clinical signs and diagnosis of feline atopic syndrome: detailed

guidelines for a correct diagnosis. Veterinary Dermatology. 32, 1 (2021), 26-e6. https://doi.org/10.1111/vde.12935.

[15] Bajwa, J. 2020. Feline dermatophytosis: clinical features and diagnostic testing. The Canadian Veterinary Journal. 61, 11 (2020), 1217-1220.

[16] Jusman, Y., Ahdan, M., Nurkholid, F., Musthofa, A., Yulianti, W., and Ramadhani, F. 2020. Expert system for detecting cat skin disease using certainty factor method. Journal of Electrical Technology UMY. 4, 1 (2020). https://journal.umy.ac.id/index.php/jet/article/view/9458.

[17] Manoj, B. S., Ganiga, P., and Vikas, H. N. 2025. Evaluating deep learning model ResNet50 for dog skin disease classification: AI-powered dog care companion. In Proceedings of the Second International Conference on Computing and Intelligent Systems (ICCIS-2025). International Research Journal of Innovations in Engineering and Technology. 9, Special Issue (May 2025), 86-91. https://doi.org/10.47001/IRJIET/2025.ICCIS-202513.

[18] Taufiqoh, S. D. and Purnamasari, P. D. 2025. Development of detection model for skin diseases in pets using image processing and deep learning techniques. International Journal of Electrical, Computer and Biomedical Engineering. 3, 2 (2025), 433-457. https://doi.org/10.62146/ijecbe.v3i2.114.

[19] Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. 2009. ImageNet: a large-scale hierarchical image database. In Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 248-255.

[20] Tajbakhsh, N., Shin, J. Y., Gurudu, S. R., Hurst, R. T., Kendall, C. B., Gotway, M. B., and Liang, J. 2016. Convolutional neural networks for medical image analysis: full training or fine tuning? IEEE Transactions on Medical Imaging. 35, 5 (2016), 1299-1312. https://doi.org/10.1109/TMI.2016.2535302.

[21] Newaz, A., Ishti, M. M., Azam, A. Z. M. A., and Adib, A. U. R. 2025. Toward accessible dermatology: skin lesion classification using deep learning models on mobile-acquired images. arXiv:2509.04800. https://doi.org/10.48550/arXiv.2509.04800.

[22] He, K., Zhang, X., Ren, S., and Sun, J. 2016. Deep residual learning for image recognition. In Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770-778.

[23] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C. 2018. MobileNetV2: inverted residuals and linear bottlenecks. In Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 4510-4520. https://doi.org/10.1109/CVPR.2018.00474.

[24] Tan, M. and Le, Q. V. 2019. EfficientNet: rethinking model scaling for convolutional neural networks. In Proc. 36th International Conference on Machine Learning (ICML), PMLR 97, 6105-6114.

[25] Kohavi, R. 1995. A study of cross-validation and bootstrap for accuracy estimation and model selection. In Proc. 14th International Joint Conference on Artificial Intelligence (IJCAI), 1137-1145.

[26] Varma, S. and Simon, R. 2006. Bias in error estimation when using cross-validation for model selection. BMC Bioinformatics. 7 (2006), 91. https://doi.org/10.1186/1471-2105-7-91.

[27] Prechelt, L. 1998. Early stopping, but when? In Neural Networks: Tricks of the Trade, G. B. Orr and K.-R. Müller, Eds. Lecture Notes in Computer Science 1524. Springer, Berlin, 55-69. https://doi.org/10.1007/3-540-49430-8_3.

[28] Chicco, D. and Jurman, G. 2020. The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. BMC Genomics. 21 (2020), 6. https://doi.org/10.1186/s12864-019-6413-7.

[29] Buda, M., Maki, A., and Mazurowski, M. A. 2018. A systematic study of the class imbalance problem in convolutional neural networks. Neural Networks. 106 (2018), 249-259. https://doi.org/10.1016/j.neunet.2018.07.011.

[30] Sokolova, M. and Lapalme, G. 2009. A systematic analysis of performance measures for classification tasks. Information Processing & Management. 45, 4 (2009), 427-437. https://doi.org/10.1016/j.ipm.2009.03.002.

[31] Gorodkin, J. 2004. Comparing two K-category assignments by a K-category correlation coefficient. Computational Biology and Chemistry. 28, 5 (2004), 367-374.

[32] Dietterich, T. G. 1998. Approximate statistical tests for comparing supervised classification learning algorithms. Neural Computation. 10, 7 (1998), 1895-1923. https://doi.org/10.1162/089976698300017197.

[33] Schenker, N. and Gentleman, J. F. 2001. On judging the significance of differences by examining the overlap between confidence intervals. The American Statistician. 55, 3 (2001), 182-186. https://doi.org/10.1198/000313001317097960.

[34] Altman, D. G. and Bland, J. M. 1995. Absence of evidence is not evidence of absence. BMJ. 311, 7003 (1995), 485. https://doi.org/10.1136/bmj.311.7003.485.

[35] Ma, N., Zhang, X., Zheng, H.-T., and Sun, J. 2018. ShuffleNet V2: practical guidelines for efficient CNN architecture design. In Proc. European

Conference on Computer Vision (ECCV), 116-131. https://doi.org/10.1007/978-3-030-01264-9_8.

[36] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. 2017. Grad-CAM: visual explanations from deep networks via gradient-based localization. In Proc. IEEE International Conference on Computer Vision (ICCV), 618-626. https://doi.org/10.1109/ICCV.2017.74.

[37] Zauner, C., Steinebach, M., and Hermann, E. 2011. Rihamark: perceptual image hash benchmarking. In Proc. SPIE 7880, Media Watermarking, Security, and Forensics III. https://doi.org/10.1117/12.876617.

[38] Hashemi, M. 2019. Enlarging smaller images before inputting into convolutional neural network: zero-padding vs. interpolation. Journal of Big Data. 6 (2019), 98. https://doi.org/10.1186/s40537-019-0263-7.

[39] Tensmeyer, C. and Martinez, T. 2017. Analysis of convolutional neural networks for document image classification. arXiv:1708.03273.

[40] Keras Team. Keras 3 API documentation: Keras Applications (ResNet and ResNetV2; EfficientNet). https://keras.io/api/applications/. Accessed September 2026.

[41] Shorten, C. and Khoshgoftaar, T. M. 2019. A survey on image data augmentation for deep learning. Journal of Big Data. 6 (2019), 60.

[42] James, G., Witten, D., Hastie, T., and Tibshirani, R. 2013. An Introduction to Statistical Learning with Applications in R. Springer, New York.

[43] scikit-learn developers. scikit-learn documentation: matthews_corrcoef, f1_score, and compute_class_weight. https://scikit-learn.org/stable/. Accessed September 2026.

[44] Kingma, D. P. and Ba, J. 2015. Adam: a method for stochastic optimization. In Proc. 3rd International Conference on Learning Representations (ICLR).

[45] Goodfellow, I., Bengio, Y., and Courville, A. 2016. Deep Learning. The MIT Press, Cambridge, Massachusetts.

[46] TensorFlow Developers. 2024. TensorFlow 2.16.1 release notes. https://github.com/tensorflow/tensorflow/releases/tag/v2.16.1. Accessed September 2026.

[47] Keras Team. Keras 3 API documentation: Model training APIs (Model.fit) and Losses (loss reduction). https://keras.io/api/models/model_training_apis/ and https://keras.io/api/losses/. Accessed September 2026.

[48] NumPy Developers. numpy.std. NumPy Reference Documentation. https://numpy.org/doc/stable/reference/generated/numpy.std.html. Accessed September 2026.

[49] Efron, B. and Tibshirani, R. J. 1993. An Introduction to the Bootstrap. Chapman & Hall, New York.

[50] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. 2009. Introduction to Algorithms (3rd ed.). The MIT Press, Cambridge, Massachusetts.

[51] Hendrycks, D. and Gimpel, K. 2017. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In Proc. International Conference on Learning Representations (ICLR). arXiv:1610.02136.

[52] O'Neill, D. G., Church, D. B., McGreevy, P. D., Thomson, P. C., and Brodbelt, D. C. 2014. Prevalence of disorders recorded in cats attending

primary-care veterinary practices in England. The Veterinary Journal. 202 (2014), 286-291. https://doi.org/10.1016/j.tvjl.2014.08.004.

[53] O'Neill, D. G., Gunn-Moore, D. A., Sorrell, S., et al. 2023. Commonly diagnosed disorders in domestic cats in the UK and their associations with sex and age. Journal of Feline Medicine and Surgery. 25, 2 (2023). https://doi.org/10.1177/1098612X231155016.

[54] Holm, S. 1979. A simple sequentially rejective multiple test procedure. Scandinavian Journal of Statistics. 6, 2 (1979), 65-70.